

孙武侦查官 · 确定性侦查推演内核

技术方案书（内部技术设计文档）

文档性质	内部技术设计文档（给研发 / 架构 / 代码审计）
依据版本	D:\dev\inves_duckdb 仓库源码逐模块核对
篇幅	8 章 · 约 42,074 字
生成日期	2026-09-11
技术栈	Python 3.12+ · DuckDB · LadybugDB（可选） · FastAPI · Vue 3

本文档全部结论以源码为唯一事实来源，未引用既有 md / txt 文档结论。
凡与既有文档冲突之处，一律以代码为准并在第 8 章记录。

孙武侦查官 · 确定性侦查推演内核

技术方案书（内部技术设计文档）

本文档基于 `D:\dev\inves_duckdb` 仓库源码逐模块核对撰写，共 8 章、约 42,074 字。

事实来源：源码本身（未引用既有 md/txt 文档结论）。凡与既有文档冲突之处，一律以代码为准并在第 8 章记录。

生成日期：2026-09-11

目录

- 第 1 章 项目定位与设计目标 （4,447 字 / 目标 4,500 字）
- 第 2 章 总体架构设计 （5,888 字 / 目标 6,000 字）
- 第 3 章 语义层: Ontology 编译内核 （6,506 字 / 目标 6,500 字）
- 第 4 章 数据接入与治理 （4,462 字 / 目标 5,000 字）
- 第 5 章 规则引擎与 Function 计算层 （5,919 字 / 目标 6,000 字）
- 第 6 章 线索生命周期、处置与写路径 （4,860 字 / 目标 5,000 字）
- 第 7 章 安全、权限与可观测性 （5,481 字 / 目标 5,500 字）
- 第 8 章 质量保障、部署与演进路线 （4,511 字 / 目标 4,500 字）

第 1 章 项目定位与设计目标

「孙武侦查官」是一个确定性侦查推演内核（编程语言 Python 3.12+，存储 DuckDB，可选图库 LadybugDB，前端 Vue 3 + TypeScript），部署形态纯离线、零 LLM 依赖、零 API Key，可运行于物理隔离网络。本章界定项目要解决的问题、系统的能力边界、用代码强制守住的三条禁令、以及贯穿全系统的六条设计原则，并给出阅读本文档所需的术语表与读者路径。本章不展开模块实现细节，那些留待第 2、3、5 章；本章只回答“我们到底在造什么、为什么这么造、哪些事我们故意不做”。

1.1 问题域：调查分析工作的三类固有困境

侦查研判在很长一段时间里是一种建立在个人经验之上的手艺。资深侦查员凭借对资金流向、通话规律、工商关联、轨迹重合的直觉，把分散在多张原始表里的线索拼成一条可追查的链条。这种手艺在实战中有效，但它有三个从结构上无法回避的困境，而这三个困境恰好也是机器能够、也应当补位的地方。

第一类困境是经验手艺不可复现。同一批数据交给两位不同的分析师，结论可能大相径庭；同一位分析师在周一和周五面对同样的材料，也可能给出不一样的判断。问题不在于人，而在于“为什么这么判”这件事没有被显式地表达出来——判据藏在人的脑子里，没有落在可重放的指令上。当一份研判报告需要向上级解释、需要被复核、需要在半年后被另案引用时，这种不可复现性就变成了追责与合规上的硬伤。本项目的核心命题，就是把“查什么、怎么判定、阈值多少”从人的脑内状态，外化成一组写在 `ontology/<pack>/*.json` 里的声明，使同样的输入永远得到同样的输出，使任何一次推演都能被完整回放。

第二类困境是多源异构数据无法对齐。真实调查材料来自银行流水、通话记录、招投标档案、工商登记、轨迹出行、公开 OSINT、举报材料七类原始来源，它们的字段命名、粒度、主键、时间口径彼此不同，且大多以中文业务表名和异构字段形态存在。在没有统一语义层的情况下，每新接入一种数据源，工程上就要重写一批针对该表结构的查询与连接逻辑，形成 $N \times M$ 的对账成本。项目以 DuckDB 温层承载中文业务冷表，并在其上编译出 `obj_*/lnk_*` 语义层（`objects.json / links.json / bindings.json`），把“数据是什么”与“数据从哪来”解耦，使规则只面向稳定的语义对象编写，而非面向每一张原始表重写。

第三类困境是结论无法追溯到原始行。调查工作的产物最终要能进法庭、能经得起质证，这意味着每条线索都必须能回答“你凭哪一行、哪一张表、哪个版本的数据得出这个结论”。如果一条线索只能说“系统算出来的”，它就不具备可采性。本项目因此把溯源做成一等公民：Function 不返回原始明细，只返回聚合结果与内容寻址的溯源 ID（`core/row_uri.py` 采用 `dataset@version#partition/rowid` 格式），需要时由 `resolve_row_uri` 回捞原始行。溯源 ID 绑定了数据集、版本与分区，使任意一条线索都能沿审计哈希链回到它诞生时刻的原始数据快照。

多源无法对齐的代价值得单独量化。项目面对的七类原始来源里，银行流水与轨迹出行以明细行粒度存在、工商登记以主体粒度存在、招投标档案以项目粒度存在、通话记录以“主叫-被叫-时刻”三元组存在，彼此没有共享主键。在没有语义层时，要把“张三在某次中标后与某账户的大额往来”连成一条链，需要手写跨表 JOIN、做人名与组织名归一（`core/entity.py` 的 `normalize_org_name / OrganizationResolver`）、做时间窗对齐，而这套逻辑对每一对新数据源都要重做一遍，这就是 $N \times M$ 对账成本的来源。语义层把这一成本一次性沉到 `bindings.json` 的管道层：类型层（`objects.json / links.json`）保持稳定，新数据源只需补一份绑定声明。

这三类困境不是孤立的。经验不可复现的根因之一是判定没被显式化；多源无法对齐放大了显式化的成本；而溯源缺失则让前两者的产出失去法律效力。项目后续所有架构决策，都可以视作对这三个根问题的回应。

1.2 系统定位与能力边界

本系统定位为“确定性侦查推演内核”，它处于调查流程的计算与提示环节，而不是决策与处置环节。更具体地说，它把多源异构原始数据编译成一套语义层，在其上执行声明式规则，产出带溯源 ID 的线索，再经去重、优先级排序、五间交叉升格，进入人工处置看板。整个链路在核心路径上不依赖任何大模型；LLM 仅作为可选增强层，承担草案生成、结果解释、字段对齐等辅助任务，且带一键降级开关与影子模式（REQ-040）。

系统的能力边界由一条纪律性命题划清：机器只做计算与提示，绝不作定性结论，绝不置“已立案”。这条边界不是写在文档里的软约定，而是在代码里由状态机与操作者校验双重强制——“已立案”被硬编码进 `core/access.py` 的 `HUMAN_ONLY_STATUSES = frozenset({"已立案"})`，任何非人操作者发起的状态迁移都会被拒绝；线索从“已固证”升格到“已立案”必须携带 `legal_basis` 且仅可由具名的人操作者触发。这样做的意图很明确：定性结论与立案决定属于人类执法权限，系统只负责把证据链算清楚、把可疑点提示出来、把每一步的依据留痕，最终是否定性、是否立案交由人判断。

在“不做什么”之外，同样重要的是“不替代什么”。系统不替代数据接入前的合法取证，不替代人工对证据真实性的质证，不替代司法程序里的任何一环。它能降低的是经验不可复现带来的不确定性、数据不对齐带来的工程成本、溯源缺失带来的合规风险。明确边界的意义在于防止系统被误用为“自动定罪”工具——这是设计上必须主动拒绝的场景。

系统把“多源 → 语义层 → 规则 → 线索 → 处置”串成一条确定性链路，并沿五技能流水线推进：庙算（MiaoSuan 假设编排）→ 知己（覆盖盘点）→ 虚实（R1-R6 规则执行）→ 奇正（时间窗碰撞等对抗性判定）→ 用间（五间交叉升格）。其中“用间”对应项目最关键的升格逻辑：单源线索只是观察，两源交叉成为线索，三源以上升级为“可立案依据候选”——这条映射硬编码不可配（仅间类名称可配），因为升格阈值一旦可被随意调整，就动摇了“线索等级”这一判定本身的客观性。最终产物进入 `DisposalBoard`（处置看板），由人完成“已固证→已立案”的闭环。

1.3 三条禁令及其代码强制点

三条禁令是前文能力边界的代码化表达。它们的共同特征是“用代码强制，而非用规范劝导”：违反禁令的调用会在运行时被拒绝，且其中两条还有静态扫描进入 CI 作为不可绕过的红线。

第一条禁令是不自己写业务 SQL。内核禁止任何业务代码直接查询七张原始业务表。强制点在 `core/store.py` 维护的 `_FORBIDDEN_TABLES = (银行流水, 通话记录, 招投标档案, 工商信息, 轨迹出行, 公开OSINT, 举报材料)`，门面层 `Store` 在 `touches_forbidden_table()` 中命中即抛 `DirectSourceAccessError`。此外 `scripts/audit_straight_sql.py` 做静态扫描，CI 测试组 `audit` 强制通过，使直查代码无法合入主干。绕过通道仅剩 `unsafe=True`，且必须带 `operator + reason` 并落入 `meta_unsafe_query` 审计表——即任何绕过都是显式、具名、可追溯的例外，而非静默的后门。这条禁令的代价是：所有面向原始数据的查询都必须先经语义层编译，接入新数据源有前置成本；收益是七张异构原始表永远不会被规则逻辑直接耦合，溯源与重建因此成为可能。

第二条禁令是不把原始明细搬进上下文。Function 只返回聚合结果与溯源 ID，绝不下发逐行原始明细，以防止敏感个人信息在推理链路中被无意义地扩散。强制点在 `core/row_uri.py` 的内容寻址机制：原始行由 `make_row_uri` 以 `dataset@version#partition/rowid` 编码，明细归档进 `row_archive / row_build_index` 表，需要回看时由 `resolve_row_uri / snapshot_source_rows` 精确回捞。换句话说，上下文里流动的是“指向数据的指针 + 聚合结论”，原始数据留在受控存储里按需取用。代价是回看原始行需要一次额外解析；收益是上下文规模可控、敏感字段不漫游、且每一份被引用的明细都能被精确锚定到版本化快照。

第三条禁令是不下定性结论、不置“已立案”。强制点有两处：其一是 `core/access.py` 的 `HUMAN_ONLY_STATUSES`，把“已立案”钉死为仅人类可写入的终态；其二是 `core/action_executor.py` 的 `FORBIDDEN_OPERATORS = {system, ai, assistant, model, bot, auto, llm}`，凡以这些身份发起写操作一律 `ValueError`。在 MCP 边界上，`clue_transition` 工具还额外拦截 `agent:` 前缀操作者，堵住“以 Agent 名义改状态”的口子。代价是系统永远无法“自

已结案”，必须由具名的人闭环；收益是定性权与立案权始终留在人类执法者手中，系统输出始终可被定性为“计算与提示”。

三条禁令彼此咬合：第一条禁止直查原始表，使所有计算流经语义层，为第二条的溯源 ID 提供稳定的命名空间；第二条限制原始明细进入上下文，使敏感数据不漫游，为第三条的“只算不判”减少越权风险；第三条把定性权锁回人类，使前两条的克制不至于在出口处被一笔勾销。它们共同构成系统的“不可自作主张”底座。

1.4 六条设计原则

六条设计原则是前三条禁令的推广，它们定义了系统在每个架构岔路口上的默认选择。每条原则都对应一个具体问题，也都附带可计算的代价。

确定性优先于智能。核心链路（语义编译、规则执行、线索去重排序、写路径）不依赖大模型；LLM 仅作为可选增强层承担草案/解释/对齐，且带一键降级开关与影子模式（REQ-040）。具体机制上，`core/llm/fallback.py` 提供一键降级并定义 `LLM_OWNED_TABLES`，降级后 LLM 相关写能力整体关闭；解释生成 `core/llm/explain.py` 内置 `_QUALITATIVE_WORDS` 定性词黑名单，防止增强层越界输出定性结论；离线环境下 `core/llm/llm_client.py` 的 `fake_invoke` 注入使链路在无 key 时仍可跑通。它解决的问题是推理结果可复现、可在物理隔离环境运行；代价是系统不擅长开放域语义理解，需要靠声明式规则覆盖能力，规则未覆盖的盲区只能交给人或降级提示。

声明是数据，实现是代码。“查什么、怎么判定、谁能看”全部写在 `ontology/<pack>/*.json` 声明文件里，Python 只提供编译器、执行器、原子能力。解决的问题是业务变更不触发代码重发、规则可被审计可回放；代价是引入一套声明式 DSL 与装载顺序的硬依赖，学习曲线与维护面转移到声明层。

类型层与管道层分离。`objects.json` / `links.json` 定义“是什么”（类型层），`bindings.json` 定义“数据从哪来”（管道层）。解决的问题是多源接入时无需为每种源重写类型定义，且类型稳定而管道可替换；代价是两个层必须保持契约一致，装载顺序有硬依赖（类型层先于管道层）。

失败要留痕，不许静默。零命中四分类、脏值降级计数、结构降级、派发失败全部进入 `run_diagnostic`，与 `findings` 物理分离。解决的问题是“没查到”与“查了没中”被区分，管线故障可见可追责；代价是诊断数据规模增长，需要独立的诊断视图而非混进结果。

未声明即拒绝（fail-closed）。对象/链接未在 `policies.json` 声明则运行时拒绝，声明文件缺失则全拒，`schema_version != 2` 在装载期硬失败（`ValueError`）。这一原则在权限面有三个具体强制点：`policies.json` 缺失时 `object_policies={}` 且 `_missing_file=True`，结果是对象级鉴权全拒；装载期 `schema_version != 2` 直接 `ValueError` 中止；声明角色集若不在 `ROLE_RANK` 内同样 `ValueError`。解决的问题是任何未显式授权的访问都不被默认放行，杜绝“配置漏写=悄悄放行”；代价是任何声明遗漏都会让对应能力整体不可用，而非降级放行——这是刻意选择的保守默认。

写路径唯一。所有写操作经 `ActionExecutor`；Function 只读，SQL 白名单强制首词为 `SELECT` / `WITH`，`_FORBIDDEN_SQL` 正则命中任何写关键字即拒。在 `ActionExecutor` 内部，一次状态迁移要过五步校验（角色占位符拦截、必填 `legal_basis`、状态机合法性、显式 `only_from` 收紧、权限上下文一致性），并经两阶段提交（submit 幂等键 `f"{action}:{clue}:{sha256(...)[:16]}"` → `approve` 必须具名 → `dispatch`，未 `approve` 抛 `NotApprovedError`）。解决的问题是写行为的可审计、可审批、可幂等；代价是任何新写能力都必须挂到 `ActionExecutor` 上，无法在旁路随意落库。

1.5 术语表

本章及后续章节频繁使用一组领域术语，其中多数是代码中的原名。为避免歧义，下表给出中文名、英文/代码原名与一句话含义；表格前后再就若干易混淆概念做补充说明。

术语	英文 / 代码原名	含义
语义层	Semantic Layer (<code>obj_*</code> / <code>lnk_*</code>)	由原始数据编译出的稳定对象/链接表，规则只面向它编写
对象	Object (<code>objects.json</code>)	类型层实体，如 person、org、transaction、call、trackpoint，含 pk 与属性
链接	Link (<code>links.json</code>)	对象间的关系边，如 transfers、calls_to、co_located，只表达关系不含判据
间类	Jian Type (<code>jians.json</code>)	五间之一：因间/内间/反间/死间/生间，带 weight 与 default_clearance
维度	Dimension (<code>dimensions.json</code>)	五维之一：资金/通讯/行为/关系/时间，规则沿维度组织
庙算	MiaoSuan (<code>core/hypotheses.py</code>)	假设引擎，做五间×五维覆盖建模与假设枚举/冲突检测
线索	Clue (<code>obj_clue</code>)	带溯源 ID 的产出，经去重/优先级/交叉升格进入处置看板
代理键	Surrogate Key (<code>ontology.py:978</code>)	语义层幂等主键，event 按行哈希、entity 按名哈希、自引用直通
降级	Degraded	计算在受限条件下完成并标 <code>is_degraded</code> ，留痕但不污染主结果
健康度	RunHealth (<code>core/run_health.py</code>)	运行期诊断，统一落 <code>run_diagnostic</code> ，与 findings 物理分离

"对象"与"链接"是类型层的两条主线，区别在于对象是有独立身份的实体、链接是实体之间的关系；规则只声明"查对象/链接表"，不关心底层原始表。"间类"容易与"维度"混淆：维度是观察线索的角度（资金/通讯等），间类是线索的来源属性与密级（因间来自工商关系、内间来自举报等），二者正交——一条线索同时有一个维度和一个或多个间类。"降级"不是一个错误状态，而是一种受控的、带标记的降级执行，它保证管线不崩、结果可解释，但明确告知读者"这条结论是在条件受限下算出的"。

"代理键"是系统可反复重建且不漂移的基石：语义层每次从原始数据重新编译时，event 型用 `sha1(json.dumps(行内容))[:12]`、entity 型用 `{name: f"{prefix}_{sha1(name)[:12]}"}` 分配主键，新增名字不改旧键，因此同一份数据反复编译得到同一套主键，溯源链才能稳定。这一点在第 3 章会展开。

1.6 文档读者与阅读路径

本技术设计文档面向工程与架构读者，不同角色关注点不同，建议按下列路径取舍，无需逐章通读。

新入职研发应先读第 1 章建立边界意识，再读第 3 章语义层与第 5 章规则/Function，理解"声明写什么、代码算什么"的分工，随后结合 `reference-keypoints.md` 的模块清单与源码对照。架构评审者应重点看第 1 章设计原则与第 2

章 ADR（六条架构决策记录），尤其关注 fail-closed 与写路径唯一这两条默认选择的取舍理由，必要时回看第 8 章已知缺陷。代码审计者必须逐条核对第 1.3 节三条禁令的强制点、第 7 章权限与可观测性，并以 `scripts/audit_straight_sql.py` 与 `guard` 测试组为入口验证红线是否被遵守。运维与部署者则先看第 1.2 节边界与第 8 章环境约束（WSL2 与 Windows 图库可用性差异、离线部署能力），再据数据流主入口 `run_all.py:47` `main()` 理解一次完整推演的产物与落盘位置。

无论哪类读者，都应将 `reference-keypoints.md` 视为唯一事实来源。本文档写作时已发现一处文档漂移：`AGENTS.md` 记载"70 组测试/73 组全绿"，而 `run_tests.py` 的 GROUPS 注册表实际为 121 组——以代码为准，凡与代码冲突的既有 md/txt 结论均让位于源码核对结果。

第 2 章 总体架构设计

本章从整体视角描述孙武侦查官（SunWu）确定性侦查推演内核的骨架：系统由“五层存储（L0 - L4）+ 四层逻辑”两个正交维度构成，前者解决多源异构调查数据的物理落盘与生命周期，后者解决声明式语义层之上的统一读、计算与受控写。第 1 章已定义语义层、对象、链接、间类、维度、庙算、线索、代理键、降级、健康度等术语，本章直接引用，不再重新解释。需要提前说明的一点是，整个内核 66 个 Python 模块（core/ 根目录 57 个 + core/llm/ 9 个）被刻意划分为九大职责分组，全部围绕“声明是数据、实现是代码”这一核心命题展开，且对外的三条产品形态（CLI / MCP / Web）在依赖方向上严格单向：它们依赖内核，内核不反向依赖它们。

2.1 架构总览：五层存储 + 四层逻辑

孙武侦查官的架构可以用两个相互正交的维度来理解。第一个维度是存储分层（L0 - L4），它回答“数据以什么形态、存在哪里、活多久”；第二个维度是逻辑分层（消费层 → 统一读网关 → 语义层 → 计算与写层），它回答“一次推理请求如何穿过系统、在哪里被管控、在哪里落地”。这两个维度正交的意义在于：无论底层是 Parquet 原始文件还是 DuckDB 温层，上层逻辑看到的始终是统一的语义层接口，存储载体的替换不应波及规则与权限。

存储侧的五层并非简单的冷热分级，而是带有明确的“数据形态跃迁”语义：L0 是原始文件（Parquet / CSV / Excel / JSON / SQLite），L1 是进程内特征缓存（`feat_l1` 表，特征集哈希驱动失效），L2 是 DuckDB 单文件温层（`investigation.duckdb`，承载中文业务冷表、`obj_*` / `lnk_*` 语义层、诊断表与审计链），L3 是不可变分区冷层（`data/*.parquet`），L4 是可选的图库（LadybugDB，`data/ladybug/*.lbug`）。逻辑侧的四层则保证：所有读请求都被 `OntologyReadGateway`（REQ-002）这一唯一入口收口，所有写请求都被 `ActionExecutor` 这一唯一入口收口，而 `AccessContext`（`frozen`、`operator` 必填）作为五条出口（网关 / Function / Action / MCP / 导出）的统一鉴权载体贯穿始终。这种“双层收口”是本系统设计稳健性的根本来源，后续 2.2 与 2.3 将分别展开。

需要强调，五层存储与四层逻辑并非各自为政，而是通过主数据流（`run_all.py:47 main()`）被串联成一条可回放的确定性链路：数据接入 → 实体解析 → 采样预演 → 语义层编译（`build_ontology`）→ 质量门扫描 → 规则执行 → 庙算 → 血缘去重 → 图计算 → 处置看板 → 产物导出。任何一个环节的健康度异常都落在 `run_diagnostic`，与线索 `findings` 物理隔离，这一点在 2.5 的 ADR-5 中有专门论证。

2.2 存储层设计（L0-L4）

存储层的核心职责是把“不可控的多源异构原始数据”逐步收敛为“可控、可审计、可回放的语义层与诊断数据”。每一层都有明确的载体、职责与数据生命周期，且层与层之间的流转存在严格条件，并非任意可达。理解这一层要先理解一个前提：本系统是纯离线、零 LLM 依赖的确定性内核，可部署于物理隔离网络，因此存储选型首先服从“单文件、嵌入式、零外部服务依赖”的硬约束，而非传统的分布式数仓思路。

L0 原始文件层是全部数据的起点，`core/data_ingest.py` 提供 Parquet / CSV / Excel / JSON / SQLite 五类适配器（`DataIngestManager.ingest_directory`），数据在此以原生格式落盘，不做任何语义改造。L1 是进程内字典加 `feat_l1` 表的轻量特征缓存（REQ-015），其键 `feature_set_hash = sha1(entity_id|time_window|feature_name|inputs)`，输入一旦变化即触发 `mark_stale`，保证特征重算的正确性而不必每次回源。L2 是真正的温层与系统重心：DuckDB 单文件 `investigation.duckdb`，既存放未做语义抽象的中文业务冷表，也存放编译产物 `obj_*` / `lnk_*`、诊断表与审计链，是绝大多数查询与规则执行的物理底座。L3 冷层为 `data/*.parquet` 不可变分区，用于快照与归档；L4 为可选的 LadybugDB 图库，未安装时全线降级为 SQL 单轨并标记 `degraded`。

为什么选 DuckDB 而非传统数据仓库？根本原因在于部署形态与确定性诉求。传统数仓（如 PostgreSQL 集群、ClickHouse、Snowflake）要求常驻服务、网络可达与运维团队，这与“可部署于物理隔离网络、零 API Key”的边界直接冲突；而 DuckDB 是进程内嵌入式分析引擎，单文件数据库随内核一起分发，无需独立服务进程，离线环境天然可用。对于本系统以读为主、批量为辅、数据规模受“案件包”边界约束（如 `default` 包 11 对象 / 10 链接、`reqd_case` 包 7 对象 / 2 链接）的负载画像而言，DuckDB 的列存向量化执行与 SQL 表达能力已足够覆盖 R1 - R6 规则与 11 个 Function 的计算需求，引入分布式数仓带来的运维复杂度与确定性回放成本远高于其收益。

为什么图库是可选而非必需？这是本系统一个重要的能力边界选择。图算法在本实现中实测仅支持两跳定长 `MATCH (a)-[e1]->(m)-[e2]->(b)`、变长跳 `MATCH (a)-[:*1..N]->(b)` 与 degree 统计（见 `server/app/graph_view.py`），而社区发现（Louvain / LPA）、PageRank、各类中心性、最短路径、k-core 均未实现。同时 LadybugDB 官方 CI 不构建 Windows 扩展，导致 Windows 原生不可用，须走 CSV 中转且 `ATTACH DuckDB` 在 Windows 下失效；在 WSL / Linux 下也需手工放置 `libduckdb.so`。鉴于图能力既薄又受平台限制，系统设计了 Cypher / SQL 双轨互校（`compare_engines` 按 `(source, bridge, dest)` 规范化键比对，金额不参与以防浮点差，不一致只标 `only_in_cypher` / `only_in_sql` 而不会自动采信任一侧），确保“无图库也能跑、有图库只是增强”。将其设为可选增强而非核心依赖，使系统在任何环境下都能以一个确定的 SQL 单轨交付结果，避免图库可用性成为推理正确性的单点风险。

五层之间的数据流转并非任意可达，而是受明确的生命周期条件约束。L0 原始文件经适配器进入后，特征缓存（L1）按需从 L2 源表计算并写入 `feat_l1`，一旦输入变化即由 `mark_stale` 使旧哈希失效，避免脏缓存被复用。L2 语义层的重建由增量规划驱动，`core/rebuild_planner.py` 的 `RebuildPlan` 以 `DEFAULT_BATCH_THRESHOLD=5000` 为批处理阈值，经 `materialize_changed` 做行级 diff 后推进版本时钟；重建前 `snapshot_source_rows` 把溯源快照写入 `row_archive` / `row_build_index`，保证事后可回捞。L2 到 L3 的流转是分区不可变归档，用于审计与历史比对，写入后不再变更。L4 图库则由 `core/graph.py` 的 `build_from_duckdb` 从语义层派生，节点必须全部导入否则边 COPY 报错，且 Windows 下因路径反斜杠被 Cypher 解析器当转义，导出路径须用 `as_posix()`。这套流转条件的核心意图是：越靠近 L0，数据越原始越不可控；越靠近 L2，数据越语义化越可审计；L3/L4 是 L2 的派生快照，永远不反向改写 L2。

2.3 逻辑层设计

逻辑层的职责是把底层的存储能力包装成“受控、可理解、可追责”的推理服务接口。它自外向内分为四层：消费层、统一读网关（OntologyReadGateway）、语义层（`obj_*` / `lnk_*`）、计算与写层。四层之间方向严格单向，外层依赖内层，内层不感知外层存在，这保证了核心推理逻辑与具体交付形态解耦——这是 2.6 讨论三条产品形态依赖方向的基础。

消费层是系统对外的所有触达点，包括 CLI 脚本、MCP Server、Web（FastAPI + Vue 3）、LLM Agent 与导出工具。它们共享同一套内核能力，但各自承担不同的信任边界与交互语义。统一读网关 `OntologyReadGateway`（`core/gateway.py`，REQ-002）是语义层的唯一读入口，任何消费方要读取 `obj_*` / `lnk_*` 都必须经它，由它施加对象级 / 链接级 / 属性级三级鉴权（见 `core/policy.py` 的 `check_object` / `check_link` / `mask_value`）并强制 `AccessContext`。这一层把“谁能看什么”从散布在业务代码中的判断收敛到一处，是 fail-closed 策略（未声明即拒绝）的物理落点。

语义层是逻辑层的圆心，由 `obj_*` 对象表与 `lnk_*` 链接表构成，是规则、Function、庙算与写路径共同操作的数据平面。它之上就是计算与写层：规则引擎（`core/rules.py` 的 `run_rules`）、Function 执行器（`core/functions.py`）、庙算假设引擎（`core/hypotheses.py` 的 `MiaoSuan`）、以及唯一的写路径 `ActionExecutor`。计算层只读语义层并产出 `findings` 与线索，写层则经 `ActionExecutor` 把人工处置结果落回 `obj_decision` / `lnk_decision_for` 等运行期表。值得注意，`OntologyReadGateway` 与 `ActionExecutor` 一进一出两个唯一入口，共同构成了“读收敛、写收敛”的双向治理，使任何一次数据的流动都可被审计哈希链（`core/audit.py`）追溯到具名 operator。

四层逻辑并非松散堆叠，而是被 `AccessContext`（`frozen dataclass`，`operator` 必填非空、`role` 默认"正兵"、`network` 限 `local / isolated / web`）这一统一载体缝合成一个受控整体。参考基线将其称为"五条出口统一管控"：网关、Function、Action、MCP、导出五处出口在放行前都必须校验同一个 `AccessContext`，且 `can_llm_call()` 在 `network="isolated"` 时返回 `False`，使离线隔离环境天然屏蔽 LLM 调用。这种缝合的意义在于，权限、人机边界与审计不是各层各自实现的副本，而是同一个 `frozen` 上下文在不同出口被反复校验的结果，从结构上杜绝了"某个出口忘记鉴权"的实现漂移。计算与写层内部也遵循只读契约：Function 经 `functions.py` 的 `_assert_readonly` 与首词 `SELECT / WITH` 校验，规则经 `rules.py` 的 `_evaluate_single` 只读语义层，任何写意图都会被写路径唯一入口拦截。于是四层逻辑在"统一上下文 + 读收敛 + 写收敛"三点约束下，成为一层可被独立验证的确定性管线。

2.4 内核模块地图（core/ 66 个模块的九大分组）

`core/` 目录下的 66 个 Python 模块（根 57 + `core/llm/` 9）按其职责被划分为九大分组，每组对应架构的一个切面。这一分组不是文档上的随意归类，而是反映了模块之间真实的依赖方向与契约边界：底层存储与连接在最内层，语义层在其上，权限面横切所有读路径，规则与函数、Action 与写路径、事件审计诊断纵向贯穿，线索假设实体承载业务语义，检测器作为独立同签名插件存在，治理与 LLM 作为可选增强挂在最外层。

A 组（存储与连接）负责一切物理 IO：`store.py` 是 L1/L2/L3 三层门面（`Store(root="data", db_path="investigation.duckdb")`），其 `touches_forbidden_table()` 直接落地第一条禁令的直查拦截；`features.py` 的 `FeatureStore` 实现 L1 特征缓存；`graph.py` 的 `GraphBackend` 封装 L4 图库与双轨互校。B 组（语义层）是本系统的计算重心，`ontology.py` 以 1380 行成为全仓最大模块，承载编译器与类型模型（`build_ontology` 见 `core/ontology.py:393`，代理键分配见 `core/ontology.py:978`），同组的 `ontology_loader.py`、`ontology_version.py`、`ontology_profile.py`、`views.py`、`derived.py`、`pack.py`、`data_map.py`、`draft_assembler.py`、`rebuild_planner.py` 分别覆盖装载、版本时钟、画像、视图、派生属性、案件包隔离、数据地图、草案生成与增量规划。

C 组（权限面）横切所有读路径：`access.py` 定义 `AccessContext` 与角色双尺（`ROLE_RANK / ROLE_CLEARANCE`），`policy.py` 的 `PolicyEngine` 实施三级鉴权，`gateway.py` 是统一读网关，`runtime_context.py` 用 `ReadOnlyStore` 屏蔽写能力。D 组（规则与函数）包含 `rules.py`、`functions.py`、`rule_dsl.py`（AST-only 不拼 SQL，`MAX_DEPTH=5`）、`threshold.py`、`clean_ops.py`（唯一清洗 op 注册表，15 个已注册 op）、`value_type.py`、`data_elements.py`、`de_recommend.py`。E 组（Action 与写路径）是写收敛的承载：`action_executor.py` 唯一写入口，`registry.py` 维护状态机与注册表，`disposal.py` 的 `DisposalBoard` 实现看板，`outbox.py / writeback.py / reconcile.py` 负责发件箱、回写与对账死信。

F 组（事件 / 审计 / 诊断）提供可观测性与留痕：`event_bus.py`（17 类事件、环检测深度 > 10）、`audit.py`（SHA-256 哈希链）、`run_health.py`（约 20 类诊断统一落 `run_diagnostic`）、`row_uri.py`（内容寻址溯源 `dataset@version#partition/rowid`）、`anomaly_channel.py`（异常线索通道，绝不参与五间交叉升格）、`metrics.py`（规则度量与推翻率告警）。G 组（线索 / 假设 / 实体）是业务语义核心：`hypotheses.py` 的 `MiaoSuan` 庙算引擎、`lineage.py` 血缘去重、`entity.py` 组织解析、`review.py / review_loop.py` 人审闭环、`deferred.py` 回捞、`case_library.py` 案例库、`object_set.py` 查询构造器、`sampling.py` 采样预演、`validate.py` 六段校验、`geo.py` 地点标准化、`ingest_validate.py` 分区校验、`search.py` 语义检索。H 组（检测器）是四个同签名 `scan(gateway, ...)` 的插件：`compliance.py`、`sensitive_scan.py`、`unit_scan.py`、`data_freshness.py`，只告警不阻断。I 组（治理 / 提案 / LLM）包含 `parameters.py`、`proposal.py` 与 `core/llm/` 下 9 个模块（`redact.py`、`guard.py`、`llm_client.py`、`draft_rule.py`、`explain.py`、`align.py`、`plan.py`、`fallback.py` 等），LLM 整体为可选增强，有一键降级与影子模式。

这九大分组本身也揭示了模块间的依赖方向：A 组（存储）被 B/C 组依赖，B 组（语义层）被 D/E/F/G 依赖，H 组（检测器）以同签名插件方式挂在 B 组之上，I 组（LLM）位于最外层的可选环。值得注意的是，分组里没有任何一组依赖 2.6 将讨论的 CLI / MCP / Web 形态——这印证了“内核零外部依赖、形态可插拔”的边界：形态的演进（例如未来新增一个桌面端）不需要改动这九组中的任何核心模块，只要新形态同样收敛到 `OntologyReadGateway` 与 `ActionExecutor` 两个入口即可。这种分组上的自治，是系统能在离线、零 LLM 环境下独立成立的结构保证，也是后续各章逐个展开每个分组内部契约时的统一前提。

2.5 关键架构决策记录（ADR）

本节以架构决策记录（Architecture Decision Record）形式固化本系统最具争议性的六项决策。每条均按“背景 → 决策 → 理由 → 代价 → 被否决的替代方案”五段式陈述，目的是为后来的维护者与审计者提供可追溯的取舍依据，而不是把当前设计包装成唯一正确解。

2.5.1 ADR-1 引入语义层而非直查业务表

背景：调查数据天然多源异构（银行流水、通话记录、招投标档案、工商信息、轨迹出行、公开 OSINT、举报材料），若规则与权限直接面向这些业务冷表编写，则会陷入 $N \times M$ 的对齐困境——每新增一个数据源或一条规则，适配成本随两者数量乘积增长，且无法在不改动代码的前提下实现跨源统一权限。

决策：所有推理逻辑只操作编译后的语义层（`obj_*` 对象表 / `lnk_*` 链接表），业务冷表仅作为语义层的输入，禁止规则与 Function 直查。第一条禁令在 `core/store.py` 以 `_FORBIDDEN_TABLES` 维护受保护表清单，命中即抛 `DirectSourceAccessError`；`scripts/audit_straight_sql.py` 静态扫描，CI 的 `audit` 测试组强制通过。仅 `unsafe=True` 旁路可绕过，但必须带 `operator + reason` 并落 `meta_unsafe_query`。

理由：语义层把“数据是什么”与“数据从哪来”解耦，规则以稳定的对象 / 链接命名空间书写，新增数据源只需补充绑定声明而不动规则；权限可基于对象级 / 链接级 / 属性级统一施加；代理键幂等分配使语义层可反复重建而主键不漂移，支撑增量与回放。

代价：引入编译链路（`build_ontology`，`core/ontology.py:393`）的额外开销与一次性的映射维护成本；绑定声明错误会在装载期或运行期暴露，问题排查多了一层间接。

被否决的替代方案：直查业务冷表（开发最直接，但 $N \times M$ 适配成本与权限碎片化不可接受）；在业务库上建视图直连（仍把语义耦合在物理表结构上，无法支撑案件包隔离与离线单文件部署）。

2.5.2 ADR-2 声明式本体（ontology JSON）而非硬编码模型

背景：侦查研判的“查什么、怎么判定、谁能看”是高频变化的部分（规则阈值、对象属性、权限策略随案件演进），若把这些固化在 Python 代码里，每次调整都触发代码改动、评审与发版，且非工程人员无法参与。

决策：把“是什么”与“判定逻辑”全部外置为声明式 JSON，八段声明（`objects.json` / `links.json` / `bindings.json` / `rules.json` / `functions.json` / `actions.json` / `views.json` / `policies.json`，`schema_version = 2`）构成本体，装载顺序有硬依赖（`load_data_elements` → ... → 组装 `OntologyPack`）。Python 只提供编译器、执行器与原子能力，“声明是数据，实现是代码”。

理由：调参、加规则、改权限均可在不触碰核心代码的前提下完成，分析师可主导演进；声明的结构化使静态校验（`validate_ontology.py --strict`）与失败留痕成为可能；`schema_version` 强制等于 2，否则装载期硬失败，杜绝半新半旧的本体进入运行态。

代价：编译器复杂度高，`core/ontology.py` 膨胀至 1380 行，成为全仓最难维护模块；声明式 DSL 存在学习曲线；装载链 13 步硬依赖，一处顺序错误会导致整包装载失败。

被否决的替代方案：Python 硬编码元数据模型（演进成本与发版耦合，非工程人员被排除在外）；ORM 映射框架（引入重依赖且无法表达“绑定 / 规则 / 权限”这类领域特有声明，反而弱化了声明即数据的约束）。

2.5.3 ADR-3 类型层 (objects/links) 与管道层 (bindings) 分离

背景：对象的“类型定义”（它本身是什么、有哪些属性）相对稳定，而数据的“接入管道”（从哪个源表、用哪段 SQL、做哪些清洗）随数据源切换频繁变化。两者若混在一个文件里，任何管道调整都会牵连类型定义，反之亦然。

决策：类型层（`objects.json` 定义 `pk / kind / name_property / properties`；`links.json` 定义端点对象与边属性，只表达关系不含检测判据）与管道层（`bindings.json` 的 `object_bindings / link_bindings`，含 `source / source_sql / clean / build_sql`）严格分离。这是六条设计原则之一。

理由：类型定义稳定可复用，管道声明可针对不同案件包自由替换而不污染类型；同一对象类型可被多个源绑定，支撑多源 UNION 与跨案复用；缺列预检、脏值降级等机制可针对管道层独立施加。

代价：分析师接入新源须同时维护两层声明，心智负担增加；两层间的契约（如 `binding` 的输出列必须与类型属性对账，否则链接物化时硬失败）需要工具保障。

被否决的替代方案：类型与管道合一的单文件声明（最直观，但失去类型复用性与管道可替换性，违背案件包隔离诉求）；完全交给我们代码内部硬编码绑定（退回 ADR-2 已否定的硬编码路径）。

2.5.4 ADR-4 写路径唯一 (ActionExecutor 是唯一写入口)

背景：线索状态、处置结论、决策依据属于高敏感写操作，且涉及人机边界（机器绝不置“已立案”）。若允许各模块直写数据库，写操作将散布在规则、看板、回写、MCP 等各处，无法统一施加状态机校验、角色校验与审计。

决策：所有写操作经 `ActionExecutor`（`core/action_executor.py`）；Function 只读，SQL 白名单强制首词为 `SELECT / WITH`。`ActionExecutor` 实施五步校验：角色必须是具名 `human`（`requires_role="human"` 且非占位 `operator`）、必填参数含 `legal_basis`、状态机 `ClueStatusMachine.validate`、显式 `only_from` 收紧、权限上下文一致性。第三条禁令以 `FORBIDDEN_OPERATORS = {system, ai, assistant, model, bot, auto, llm}` 强制人机边界，MCP `clue_transition` 额外拦截 `agent:` 前缀。

理由：写收敛使每一次状态变更都经过同一套校验与两阶段提交（`submit` → `approve` → `dispatch` → `mark_confirmed`，REQ-012），幂等键 `f"{action}:{clue}:{sha256(...)[:16]}"` 防止重复；与审计哈希链、`AccessContext` 五出口管控天然对齐；派发不可用时 `fail-closed` 为 `dispatch_failed` + `critical` 诊断，不假装在途（REQ-G-020）。

代价：写能力收敛牺牲了局部灵活性，新写场景必须先定义 `ActionSpec` 再走网关；两阶段提交引入 `propose / approve` 的人工环节，自动化的即时写被刻意排除。

被否决的替代方案：各模块直写 DuckDB（最灵活但无法统一审计与人机边界，违背第三条禁令与写路径唯一原则）；在 Function 内允许写（被只读红线 `_FORBIDDEN_SQL` 正则与 `_assert_readonly()` 首词校验直接否决）。

2.5.5 ADR-5 诊断与 findings 物理分离

背景：推理过程会产生大量健康度信号——零命中四分类、脏值降级计数、结构降级、派发失败等。若把这些诊断混入线索 `findings`，会污染“线索真实性”，使审计者无法区分“系统告诉我们发现了什么”与“系统告诉我们过程哪里不健康”。

决策：诊断数据统一落入 `run_diagnostic`（由 `core/run_health.py` 的 `RunHealth` / `NullRunHealth` 驱动，约 20 类 `KINDS`），与 `findings` 在物理上分表存放。降级协议（REQ-G）统一约定“降级但不静默，留痕但不污染”。`health=None` 时退化为 `NullRunHealth` 空操作。

理由：物理分离保证了线索集合的纯净——`findings` 只反映规则命中与实体关联，不受过程噪声干扰；诊断可独立查询、独立告警而不影响研判结论；与审计链、事件总线共同构成可观测性底座（`core/audit.py` / `core/event_bus.py`）。

代价：需要维护两套视图与两套生命周期，导出与看板须分别读取；消费方须理解何时看 `findings`、何时看 `diagnostic`，认知成本上升。

被否决的替代方案：同表加一列 `is_diagnostic` 标记（看似简单，但查询与导出极易误把诊断当作线索，且无法从存储层面保证审计纯净）；把诊断丢弃只保留异常（违反“失败要留痕，不许静默”的设计原则）。

2.5.6 ADR-6 图库作为可选增强而非核心依赖

背景：资金过桥、账户网络、两跳过桥等侦查场景天然适合图计算，但本实现图能力实测仅覆盖两跳定长、变长跳与 `degree`，且 `LadybugDB` 在 `Windows` 原生不可用，算法能力薄、平台依赖强。

决策：L4 图库（`LadybugDB`，`data/ladybug/*.lbug`）设为可选增强层。未安装时 `available=False`，全线降级 `SQL` 单轨并标记 `degraded`；已安装时走 `Cypher` / `SQL` 双轨互校（`compare_engines`），不一致只标注不采信。图计算仅作为确定性 `SQL` 单轨的旁证，而非结论来源。

理由：把图库降为可选，使系统在任何部署环境（含物理隔离的 `Windows` 节点）都能以一个确定的 `SQL` 单轨交付结果，避免图库可用性成为推理正确性的单点；双轨互校在不信任任一侧的前提下提供额外置信，符合确定性优先原则。

代价：失去社区发现、`PageRank`、中心性、最短路径、`k-core` 等高级图分析能力，复杂网络拓扑研判须以多跳 `SQL` 近似；平台限制使 `Windows` 下无法启用原生图加速。

被否决的替代方案：将图库作为核心存储与计算底座（会令 `Windows` 环境完全不可用，且薄算法能力成为系统正确性瓶颈）；完全放弃图能力（损失两跳过桥等直观研判手段，故选择“可选增强 + 双轨互校”折中）。

2.6 三条产品形态与职责边界

孙武侦查官对外交付为三条独立的产品形态，但它们共享同一套内核能力，且在依赖方向上严格单向：`CLI`、`MCP`、`Web` 都依赖 `core/`，`core/` 不反向依赖任何一条形态。这种单向依赖是“内核确定性、形态可插拔”的保证——形态的增删不影响推理正确性，内核的演进也不必为形态妥协。

`CLI`（命令行）是 22 个 `scripts/` 脚本构成的离线操作面，覆盖从 `init_duckdb.py`（从 `ontology` 反推中文冷表 `DDL`，且不 `import core`）、`build_ontology.py`、`build_all.py`、`incremental.py`（季度增量）、`validate_ontology.py --strict` 到 `export_dashboard.py` / `export_ladybug.py` 等长尾工具，是内核最直接的调用者，面向工程与批处理场景。`MCP Server` 是 13 个工具（`scripts/mcp_server.py` 的 `_TOOL_IMPL`）构成的手写 `JSON-RPC 2.0 over stdio` 服务（协议版本 2025-06-18，零第三方依赖），是 `Agent` 接入本系统的唯一通道；其中 `review.submit_proposal` 是 `Agent` 唯一写通道，注入扫描 → 白名单清洗 → 状态变更拦截 → 建 `status=draft` 提案，永不自动生效，所有返回体强制挂 `needs_human_review` / 定性_policy / `disclaimer`。`Web`（`FastAPI` + `Vue 3` + `TypeScript`）是人工处置看板的最终承载，以 `DuckDB` 处置表为真值源（`scripts/export_dashboard.py` 覆盖 `JSON` 快照）。

三条形态的职责边界可概括为：`CLI` 负责“构建与运维”，`MCP` 负责“`Agent` 受控读写”，`Web` 负责“人工研判与处置”。它们的写能力最终都汇流到 `ActionExecutor`，读能力都经过 `OntologyReadGateway`，因此无论哪种形态触发，数据流动

的治理闸门（权限、审计、人机边界、诊断分离）都一致生效。值得特别指出，MCP 的 `clue_transition` 与 `review.submit_proposal` 在人机边界上比 Web 更受约束（Agent 写通道只产生 draft 提案、禁止 `agent:` 前缀操作人），这是因为 Agent 形态绕过了具名人工操作员的直接在场，必须用更紧的 fail-closed 来补偿信任缺口。

第 3 章 语义层: Ontology 编译内核

第 1 章已经把"语义层"、"对象 (obj_*)"、"链接 (lnk_*)"、"代理键"、"降级"等核心术语与三条禁令、六条设计原则交代清楚,第 2 章则给出总体架构与 ADR-1/ADR-2/ADR-3 (引入语义层、声明式本体、类型与管道分离)。本章不再论证"为什么要有语义层",而是把镜头对准编译器本身,回答一个更具体的问题:这套 obj_*/lnk_* 语义表,究竟是怎么从一个目录下的 JSON 声明被编译出来的。语义层不是数据库里的静态表,而是一段可重复执行、可回滚、可增量重建的确定性计算过程,其全部"查什么、数据从哪来、怎么判定、谁能看"都写在 `ontology/<pack>/*.json` 里,Python 只提供编译器 (`core/ontology.py`)、装载器 (`core/ontology_loader.py`) 与执行器,不承载任何业务 SQL。理解编译内核,是理解后续规则引擎、写路径、权限与增量重建的前提。

3.1 为什么需要语义层

多源异构数据进入侦查内核时,最大的隐性成本不是"把数据读进来",而是"让每一种消费方都能正确读懂每一种数据"。在孙武侦查官的真实案件包里,原始数据天然横跨七类来源:银行流水、通话记录、招投标档案、工商信息、轨迹出行、公开 OSINT、举报材料(这七类也正是三条禁令中 `_FORBIDDEN_TABLES` 所列的直查拦截对象)。而语义层在 `default` 包中要向上提供 11 类对象 (`person`、`org`、`account`、`transaction`、`call`、`trackpoint`、`bid_project`、`clue`、`decision`、`tipoff`、`osint_article`)。设想没有语义层、由规则与检测器直接读原始表:每一条规则 (R1 - R6) 为了计算"资金""通讯""关系"等概念,都必须各自理解这七类源表的列约定、编码习惯与口径差异,这就构成了典型的 $N \times M$ 问题——七个源模式乘以十一类对象语义,理论上需要维护 77 组"源表→概念"的映射,且每新增一类数据源,所有消费方都要被改一遍。

语义层把这个交叉积拆成了两段单向依赖:第一段是"源表→对象"的一次性绑定,由 `bindings.json` 声明、编译器执行,只做 N 次;第二段是"对象→规则"的消费,规则只说 obj_*/lnk_* 的词汇,完全不接触原始列名,只做 M 次。 $N \times M$ 因此被压成 $N+M$ 。一个具体的对照是:R4「二人公示期轨迹同框」在 `rules.json` 里只写"直接 SELECT `lnk_co_located` (不同主体同地点 ± 1 天)",它完全不必知道轨迹出行原始表叫什么、经纬度列叫什么;银行流水、工商信息等各自如何变形进 `obj_transaction` / `obj_org`,是 `bindings` 一次性解决的问题。若去掉语义层,这条规则就得自己解析轨迹源表的 schema,七个源模式就乘以六条规则,维护成本与出错概率同步放大。除此之外,语义层还顺势解决了三个没有它就无法低成本解决的问题:其一是统一类型系统(原始 CSV 里的"金额"可能是字符串,进入 `obj_transaction` 必须落为 DOUBLE,见 3.6);其二是统一代理键(原始表没有稳定主键,语义层用内容哈希生成,见 3.5);其三是 fail-closed 的授权边界(未声明即拒绝,见 3.2)。换句话说,语义层本质上是把"数据对齐"这件最容易出错、最容易被写成散落各处的临时胶水代码的事,收口成一个可审计、可重放的编译步骤。这正是 ADR-1 引入语义层、ADR-2 采用声明式本体的根本动机。

3.2 八段声明模型 (schema_version = 2)

语义层的全部"是什么"与"从哪来"由一组 JSON 声明描述,这些声明被划分为八个带 `schema_version = 2` 标记的段落,各自承担单一职责。类型层只回答"是什么":`objects.json` 声明对象的主键(pk)、类型(entity 或 event)、身份/展示属性(name_property)与属性→值类型的映射,刻意不包含任何数据来源信息;`links.json` 只表达对象之间的关系与边属性,只表达关系、不含任何检测判据。管道层回答"数据从哪来":`bindings.json` 通过 object_bindings (source / source_sql / clean / optional) 与 link_bindings (build_sql) 把类型层挂到具体数据源上。规则手册 `rules.json` 用自然语言 `rule_text` 写判据,并挂唯一的机器执行钩子(function / params);能力目录 `functions.json` 给出这把钩子的实现,SQL 实现强制 SELECT/WITH 白名单、参数用 `{{param}}` 占位,Python 实现需注册进 `FUNCTION_IMPLS`。写路径 `actions.json` 声明角色、必填参数、副作用与终

态，其 `allowed_from` 由 `states.json` 反向派生而非手写。语义视图 `views.json` 按角色投影、查询时派生不物化；权限面 `policies.json` 给出对象级/链接级策略与属性级敏感列遮蔽。

八段声明的分层不是文档约定，而是被装载机强制的硬契约。装载机 `load_pack` 在读取 `objects.json` 前就要求 `schema_version == 2`，一旦不符立即抛 `ValueError` 使装载期硬失败，不存在任何灰度兼容。这套分层同时贯彻了“类型层与管道层分离”的设计原则：`objects.json` 里找不到 `source` 字段，`bindings.json` 里找不到判据字段，两者通过对象名在装载期拼接，任何越界声明都会在后续对账中被发现。

与八段声明平行存在的，是一组非八段、不被编译进 `obj_*/lnk_*` 的配置/知识文件。它们同样以 JSON 存放于 `ontology/<pack>/`，但不参与语义表物化，而是喂给其它子系统：`thresholds.json`（阈值策略）、`dimensions.json`（五维声明）、`jians.json`（五间声明）、`states.json`（状态机）、`enum_space.json`（枚举空间）、`scoring.json`（评分）、`derived_properties.json`（派生属性）、`data_elements.json`（数据元标准）、`llm_policy.json`（LLM 治理）、`clean_rules.json`（案件级清洗词表）。其中尤其值得强调 `case_knowledge.json`——它是人名唯一合法存放处，检测器与分析逻辑一律不得把自然人姓名硬编码进代码（CI 静态扫描 `scan_hardcoded_names.py` 兜底），破案主体身份只能源于这份声明。把“知识”与“声明”分开，是为了让编译内核保持稳定：新增一个枚举值、补一份评分配置，都不应触发语义表的重建或破坏八段 `schema` 的校验。

权限面的 `fail-closed` 是八段模型不可绕过的强制点。`policies.json` 中未声明的对象或链接，运行时一律拒绝访问；文件本身缺失则 `_missing_file=True` 导致全拒。这与第 1 章“未声明即拒绝”的设计原则一致，也意味着接入新数据源的第六步必须补 `policies.json`（见参考基线第 16 节），漏写不会静默放行，而是被运行时拦死。

3.3 装载流程与硬依赖顺序

声明文件只是静态输入，真正进入编译前要先被 `load_pack`（`core/ontology_loader.py:102`）组装成一个内存中的 `OntologyPack` 对象。这一步不是简单地按文件名顺序读取，而是一条有严格硬依赖的 13 步装载链，顺序背后的依赖关系决定了某些文件必须先于另一些文件读取：数据元标准 `load_data_elements` 必须先于对象装载，因为 `objects.json` 里属性的 `data_element` 引用需要校验该 ID 已在数据元注册表登记（`data_elements.json` 缺失则回落 `_shared` 包基线）；敏感属性遮蔽集 `_load_property_mask_set` 必须先于对象，确保敏感 `data_element` 已声明遮蔽；五间声明 `load_jians` 必须先于对象与链接，因为对象/链接上的 `jian` 字段必须已在 `jians.json` 声明；状态机 `load_states` 必须先于 `actions.json`，因为 `action` 的 `target_status` 必须落在已声明状态集内；维度声明 `load_dimensions` 必须先于 `rules.json`，因为规则的 `dimension` 必须是已声明维度（缺省回落内置五维）。其余依次是 `_load_objects` → `_load_links` → `_load_bindings` → `_load_actions` → `_load_functions` → `_load_rules` → `load_enum_space` → `_load_clean_rules`，最终组装 `OntologyPack` 并写入缓存。

这 13 步的硬性顺序不是性能优化，而是正确性约束：若 `objects.json` 先于 `jians.json` 装载，装载机无法校验 `jian` 字段合法性，只能把错误推迟到编译或运行时才暴露，那便违背了“装载期尽量早失败”的工程取向。整个链条的单点失败都会向上抛异常，不存在“缺一块也能凑合跑”的降级路径——这正是对声明式本体可信度的保障。

装载机还引入了一层指纹缓存以避免重复解析全部 JSON。入口处 `_pack_fingerprint` 计算声明目录的指纹（基于文件 `mtime`/内容），与 `pack`、`base_dir` 共同构成 `cache_key`，命中 `_PACK_CACHE`（`core/ontology_loader.py:44`，模块级字典）则直接返回缓存的 `OntologyPack`，跳过整条 13 步解析。缓存失效的判据就是指纹变化：任何一个声明文件被修改，指纹随之改变，下一次 `load_pack` 必然重新装载。这一机制对增量重建尤为关键——同一进程内反复调用 `build_ontology` 时，声明未变则复用缓存对象，声明变了则强制刷新，从而保证“编译结果”与“声明内容”严格对应，杜绝了陈旧缓存导致的诡异不一致。需要注意的是，缓存只加速“声明→`OntologyPack`”的解析阶段，不影响“`OntologyPack`→`obj_*/lnk_*`”的物化阶段；物化阶段的正确性由版本时钟（3.7）独立保障，二者职责分离，互不掩盖错误。

另外一个常被忽略的细节是：指纹缓存是进程内的模块级字典，不跨进程、不落盘。这意味着如果两个独立进程（例如 Web 服务与 CLI 脚本）同时运行，它们各自维护各自的缓存，不会因共享内存而产生竞态；代价是同一份声明在两个进程里会被各解析一次。对于离线单进程的侦查内核而言，这种取舍是合理且安全的——它把“缓存一致性”这个难题降级为“单进程内幕等”，而非引入跨进程锁。

3.4 编译流程

装载得到 `OntologyPack` 后，`build_ontology`（`core/ontology.py:393`）执行真正的物化。它的整体流程可分成六个阶段，对外返回 `{"objects": {...行数}, "links": {...行数}, "skipped": [...], ...}` 的统计字典。

第一阶段在事务之外准备上下文：`load_pack` 拿到声明后，先算 `_default_org_names` 并调用 `build_clean_context` 汇成清洗上下文（`clean_rules.json` 合并/替换内置基线词表，对应 REQ-D-007），再 `_load_entity_mapping` 读入 REQ-016 受保护的归并映射。第二阶段进入 `BEGIN TRANSACTION` 的原子块：逐个非 runtime 对象执行 `_compute_object_rows` → `_create_obj_table` → `_insert_object_rows`；随后逐个非 runtime 链接执行 `CREATE TABLE lnk_<name> AS <build_sql>`。链接物化后有一次列对账——编译器取出 `lnk_*` 的实际输出列集合，与 `links.json` 声明的边属性逐一比对，任何声明属性不在输出列中即抛 `ValueError` 硬失败（见 `core/ontology.py:448-455`，注释明确写“不准带病编译”）。整段 Object+Link 物化要么全部成功 `COMMIT`，要么在异常时 `ROLLBACK`，保证语义层不会出现“对象建好但链接残缺”的中间态。

第三阶段 `ensure_runtime_tables`（`core/ontology.py:496`）按类型声明为 runtime 对象/链接建空表（`CREATE IF NOT EXISTS`，不碰既有数据），典型如 `obj_decision`、`lnk_decision_for`——它们不由编译器填数据，只在 `ActionExecutor` 副作用写路径时被插入。第四阶段 `ViewMaterializer.materialize_all(skip_missing_base=True)` 物化 Object Views（REQ-046）：`skip_missing_base=True` 使得若某视图的基对象因 optional 而未建表，则跳过该视图而非拖垮整次 BUILD，与链接编译失败跳过的口径一致。第五阶段 `compute_version` + `record_version`（`core/ontology_version.py`，REQ-001）为本次构建写入一条 `is_current=true` 的版本记录，构成版本时钟。第六阶段 `snapshot_source_rows`（`core/row_uri.py`，REQ-008）把源行做内容寻址归档，分区标记为 `BOOTSTRAP_PARTITION`（全量启动，不继承旧索引），使任何一行 `obj_*` 都能回溯到原始明细行。这里值得停下来理解事务边界的取舍：Object 与 Link 的物化被包在单一事务里，但 runtime 表、Views、版本时钟、溯源归档都发生在 `COMMIT` 之后。这不是疏漏，而是刻意的层次划分——原子性要保障的是“语义表本身要么全成、要么全无”，而版本时钟与溯源是“本次成功构建的副产品”，它们建立在已提交的一致快照之上。下游的统一读网关 `OntologyReadGateway`（REQ-002）读取的永远是某个已 `COMMIT` 的完整语义层，不会看到半成品；版本时钟与溯源归档即使在本阶段之后出错，也只影响可观测性，不污染语义表本体。这种“核心物化强原子、周边产物弱依赖”的分层，使编译内核在异常时既不会留下脏表，又不会因为溯源写失败而把整个 BUILD 判废。

关于 runtime 表还需点明一点：它们虽不承载编译数据，却是写路径的“落点”。`obj_decision` / `lnk_decision_for` 由 `ActionExecutor._create_decision` 在分析师做出 `file`（固证/立案）类副作用时插入，编译器只负责“把空表先建好、把 schema 锁定”。这保证了无论写路径何时触发，目标表一定存在且列序稳定——把“建表”与“写数据”分离，也意味着 runtime 表的 schema 受语义层版本管理，而非散落在 `ActionExecutor` 的临时 DDL 里。

3.5 代理键的幂等分配

这是本章最关键、也最容易被低估的机制。语义层能否“反复重建而主键不漂移”，直接取决于代理键（surrogate key）的分配方式——而这一点决定了历史线索、审计链、跨案引用能否始终对得上行。

设想一种最朴素的方案：按插入顺序给每个实体编 `person_0001`、`person_0002`……这种方案在首次构建时没问题，但一旦源数据新增一个名字并重新构建，所有后续序号都会整体后移一格，`person_0005` 可能从“张三”变成“李四”。语义层之

上沉淀的每一条线索、每一次审计记录、每一处外键引用，都会因此指向错误的主体。对侦查系统而言这是灾难性的：历史研判结论与当下数据发生静默错位，且完全无迹可寻。孙武侦查官因此采用内容哈希式代理键：键只由内容决定，与插入顺序、与其它名字的存在与否都无关。

代理键的分配由 `_compute_object_rows`（`core/ontology.py:978`）在清洗、归并、业务键去重之后统一调度，具体落在 `ontology.py:1095-1114` 的分发逻辑，键的哈希实现由 `_proxy_keys`（`ontology.py:354`）与 `_event_proxy_keys`（`ontology.py:365`）两个助手完成。它按对象类型分三条策略：

其一是 `event` 型（`transaction` / `call` / `trackpoint` 等过程性事件）。事件没有稳定“名字”，编译器把每一行内容排序后取 `sha1(json.dumps(行内容))[:12]`，前缀 `prefix = otype.pk.split("_")[0]`（如 `txn_`、`call_`）。内容完全相同的多行（如同一天同一金额的两笔）按确定性顺序追加 `_02` / `_03` 后缀保证逐行唯一。这里有一处精细处理：`exclude_idx` 会排除 `composite`（复合）属性列的索引（REQ-D-013 AC-5），使得“仅备注/拆分标记不同”的两行仍被视为同一事件，不会因复合列差异而裂变出不同主键。

其二是 `entity` 型（`person` / `org` / `account` 等主体）。它按 `name_property` 的值排序，对每一个名字算出 `{name: f"{prefix}_{sha1(name)[:12]}"}`。这是严格幂等的：一个名字永远映射到同一个键，后续新增任何别的名子都不会改变既有名字的键。这正是 REQ-004 增量重建“只重写受影响行”的前提——若键随插入顺序漂移，增量更新就不可能只改写局部。

其三是自引用对象（`pk` 等于 `name_property`，例如 `clue`）。它直接直通源自然键，不做重映射，因为 `runtime` 链接要按这个自然键关联。

需要特别强调分配时机：代理键是在“实体归并映射（REQ-016, `variant`→`canonical` 折叠）”与“业务键去重（REQ-D-015）”之后才生成的。归并映射把分析师确认过的别名变体折叠到同一 `canonical`，去重按业务键剔除重复导入，二者都发生在键分配之前，因此合并后的变体行会共享同一个代理键，不会出现“同一个真实的人有两个主键”的分裂。这一点对侦查内核尤为致命：如果“张卫国”与“老张”被识别为同一主体却拿到两个代理键，那么分别挂在两个键下的资金线索、通话线索将无法在图谱层汇合，五间交叉升格（第 7 章）会漏算来源数，直接动摇“可立案依据候选”的判定。幂等键因此不只是一个工程整洁度问题，而是多源线索能否正确汇聚的前提。

`prefix` 的取法同样有讲究：`prefix = otype.pk.split("_")[0]`，即取主键名第一段。例如 `obj_person` 的 `pk` 是 `person_id`，前缀就是 `person_`，生成的键形如 `person_a1b2c3d4e5f6`。这种前缀让代理键在肉眼可读的同时保持命名空间隔离——`person_` 与 `org_` 永远不会撞键，排查数据时一眼能看出键属于哪类对象。它与审计哈希链（第 7 章 `core/audit.py` 的 SHA-256 链）形成呼应：审计链保证“操作不可篡改”，幂等键保证“被指向的主体稳定不变”，二者共同让“某条审计记录指向某人”这个事实在反复重建后依然成立。这条链路环环相扣：幂等键 → 增量只改受影响行 → 历史外键不漂移 → 审计链与线索可回放。没有它，语义层就只能“构建一次、不可重建”，而“可反复重建”恰恰是孙武侦查官把侦查变成确定性计算过程的根基。

3.6 类型系统与脏值降级

语义层面对的原始数据天然脏、乱、缺。编译内核的设计取向非常明确：脏值可以降级，但 `BUILD` 绝不中断。这意味着任何单行的类型错误、缺列、坏值，都不应导致整张 `obj_*` 表编译失败，而是被分类、留痕、降级处理，与 `findings` 物理分离地进入诊断（对应设计原则“失败要留痕，不许静默”）。

类型系统的核心是 `TYPE_SQL` 映射，把声明里的值类型翻译成 DuckDB 列类型：`string`→`VARCHAR`、`integer`→`BIGINT`、`decimal`→`DOUBLE`、`date`→`DATE`、`boolean`→`BOOLEAN`、`timestamp`→`TIMESTAMP`、`duration_days`→`INTEGER`、`enum`→`VARCHAR`、`json`→`VARCHAR`。`objects.json` 里每个属性都标注值类型，编译器据此建表与转换。

转换阶段对坏值采用 `TRY_CAST`，其处理存在三态选择，由 `binding` 的 `on_cast_error` 与对象属性配置决定：默认态是"降级为 NULL"，并把 `obj_x.prop<-raw: N 行不可转 t` 这样的计数记进统计，供诊断回溯，BUILD 继续；当某属性显式声明 `on_cast_error=fail` 时，转换失败升级为硬失败，整次构建中止——用于那些"类型错了就毫无意义"的关键列；第三种态是 `quarantine`（隔离），整行被剔除并落入 `build_quarantine` 表，同时追加隐藏列 `__raw_<alias>` 保留原始值用于事后判定，既不污染主表、也不丢失证据。`reqd_case` 包里的 `cp_whole`（composite 整列降级）、`call_old`（缺对端列降级）正是这条路径的验证样本。

对于"缺列"这一特殊脏值，编译器区分两种预检结果：`raw=None` 即源表根本无该列时，渲染为 `CAST(NULL AS TYPE)`，保持 `obj_*` 的列集与列序稳定，不破坏下游 schema；而列级预检在编译 SQL 前先用 `DESCRIBE` 检查源表——声明为必填的列缺失则硬失败，且报错直指缺失列名（不再裸抛 `BinderException`）；声明在 `optional_columns` 里的可选列缺失则降级为类型化 NULL 并留痕（`source_column_missing`）。

三态选择在实践中并非随意配置，而是按"该列对判定有多关键"逐属性决定。金额、日期这类驱动规则计算的核心列，通常用 `on_cast_error=fail`——转不出来说明源数据根本不可用，宁可中止也不产出似是而非的零值；姓名、备注等展示性或辅助性列用默认降级 NULL，保证主表不空；而那些"整行都不可信"的场景（如关键复合字段整体损毁）才动用 `quarantine`，把整行隔离保留而非丢弃。这种分级让编译器在"宽松到丢数据"与"严格到一碰就挂"两个极端之间取得了可声明、可审计的中间态，也呼应了参考基线中"降级但不静默、留痕但不污染"的统一降级协议（REQ-G 系列）。此外，清洗 op 链经 `compile_sql_expr()` 注入，且位于 CAST 之前——先清洗、后转型，避免脏格式直接进入类型转换。

多源场景另有专门的裁剪逻辑。当一个对象由手写多源 `UNION`（如 `source_sql` 跨六类源表聚合 `obj_person`）声明时，若只导入了其中部分源表，编译器通过 `_prune_union_sql()` 按"已挂载的源表"剔除引用缺失表的分支后再编译，避免 `CatalogException` 拖垮整次 BUILD；全部分支都缺失时才按"源表缺失"同口径跳过。这条路径把"部分接入"变成了受控降级，而不是编译失败。

3.7 版本时钟与增量重建

语义层不是一次性产物，它需要随源数据更新而演进，但不应每次都全量重算。支撑这一点的，是 `core/ontology_version.py`（REQ-001）实现的版本时钟与 `core/rebuild_planner.py` 规划的增量影响面。

版本时钟的核心是双 watermark，它们各自回答一个不同的问题。源端 watermark 取自"含日期/date/time 列的源表"的 `MAX` 时间，回答"源数据是不是已经变了"；语义层 watermark 记录上一次成功构建的时间点，回答"语义层是不是还没跟上源端"。两者比较得到状态：源端前进而语义层落后 → `STALE` 并附带 `affected_objects` 清单；从未构建过 → `UNBUILT`。把两个问题分开，是为了让调度方精准判断"该不该重建"以及"重建哪些对象"，而不是笼统地全量重跑。

除了时间 watermark，编译器还维护两份内容哈希以支撑指纹级判断：`_compute_input_hashes()` 对每个对象按 pk 排序后取全行 JSON 的 sha256，反映"输入数据变了没"；`_compute_params_hash()` 则把规则参数、函数参数、以及 views 声明一并哈希，反映"声明/参数变了没"。任一哈希变化都触发相关对象的重建。把 views 声明也纳入参数哈希是一个容易被忽略但很重要的细节：视图投影属于"语义层对外呈现的契约"，视图声明变了意味着下游读到的列子集变了，理应触发重建而非沿用旧版本时钟——否则版本号前进了，视图契约却没更新，会出现"版本说已最新、实际投影已过期"的错位。

双 watermark 与双哈希合起来回答四个独立问题：源端时间是否前进（源 watermark）、语义层是否落后（层 watermark）、输入数据是否变化（输入哈希）、声明参数是否变化（参数哈希）。调度方据此区分"全量重建"与"局部增量"，避免在数据纹丝未动时做无用重算，也避免在声明微调后误判为"无需重建"。这种把"变没变"拆成可独立观测的多个维度的做法，是增量重建既快又不出错的基础。

实际的增量执行由 `materialize_changed(conn, plan)` 完成，其顺序是对象先于链接（链接依赖对象键），过程为：先把受影响对象物化进 TEMP 的 `_STAGE` 临时表，再用 `_diff_apply()` 做行级 diff（只把增/删/改的行应用到正式

obj_*），随后推进版本时钟、执行 `snapshot_source_rows` 溯源归档，并发出 `ontology.materialized` 事件通知下游。行级 diff 的粒度意味着一次增量只动变化的行，这又回扣到 3.5 的幂等键——正因为键由内容决定，diff 才能稳定地按主键对齐新旧行。

增量重建有一类特殊保护对象：`entity_mapping`（REQ-016 受保护归并映射）不被重建覆盖，避免归并结果被冲掉。批处理规模由 `rebuild_planner.py` 的 `RebuildPlan` 与 `DEFAULT_BATCH_THRESHOLD=5000` 控制，超过阈值的增量任务会被分批，防止单次事务过大。

3.8 Object Views 与 DerivedProperty

语义层对外暴露的不只有物化的 `obj_*/lnk_*`，还有两类“查询时派生、不物化”的投影：Object Views 与 DerivedProperty。它们的共同设计哲学是——能算出来的，就不存下来，从而避免物化数据与派生数据之间的漂移。

Object Views 由 `core/views.py`（REQ-046）实现，声明在 `views.json`，被 `ViewMaterializer` 编译为 DuckDB 的 `v_<name>` 视图。视图本质是 `SELECT <属性子集> FROM obj_<base_object>` 的 SQL 投影，只引用基对象的列子集，不复制数据、不引入新数据来源。标准视图还会自动生成：每个对象一套 `v_<obj>_basic`（pk + name_property）与 `v_<obj>_full`（全部属性），均全角色可见，细粒度权限仍交给 `PolicyEngine` 在读时执行，不在视图层重复声明。这一点的安全含义很重：视图只是投影，权限事实仍按 `base_object` 判定——读视图时仍经 `PolicyEngine.check_object` 与 `apply_row_masks`（对象级 + 属性级遮蔽），视图本身不持有任何“谁能看什么”的事实。更进一步，`v_*` 不进入 `Store.query` 的安全表白名单，任何绕过 `OntologyReadGateway.view()` 直查 `v_*` 的行为都会被视作越权嫌疑。视图定义本身也在装载期纳入 schema 校验，`base_object` 引用必须存在、`properties` 必须是对象属性子集、`roles` 必须落在 `ROLE_RANK` 内，任一不一致硬失败。

DerivedProperty 由 `core/derived.py`（REQ-028）实现，与 Object Views 同源思路：属性在查询时按需派生，而非预先物化进表。`_FORBIDDEN_REGISTRY_NAMES` 明确禁止把“评分（scoring）”之类动态计算结果塞进静态注册表——评分不是客观存在的事实，一旦物化就会与实时计算脱钩、产生错误的“既成事实”印象。把这类派生属性限定在查询时计算，既保证了结果随时与最新数据一致，也守住了“机器只做计算、绝不置定性结论”的边界。

两类机制合在一起，构成了语义层“物化稳定事实、派生动态视图”的双层结构：`obj_*/lnk_*` 承载经过类型校验与代理键对齐的硬事实，Views 与 DerivedProperty 承载面向角色、面向场景的软投影，二者在查询入口汇合于统一读网关，既隔离了底层脏数据，又不让派生层成为新的数据漂移源。

把 Object Views 与 DerivedProperty 放在同一节对照，是因为它们常被混淆，却有不同适用边界。Object Views 解决的是“同一份对象、不同角色看不同列”的投影需求，本质是列的子集与角色过滤，挂在 `views.json` 声明、编译为数据库视图；DerivedProperty 解决的是“某属性本身就不是源数据、需要按公式算出来”的需求，例如一个对象的衍生风险分值，它不属于任何源表，也不应被当作既成事实物化。二者的共性是不物化、查询时现算、权限仍回退到 `base_object`；差异在于前者是“已有列的裁剪投影”、后者是“按需计算的新列”。把这两者从 `obj_*/lnk_*` 物化流程中剥离出来，还带来一个架构上的好处：调整一个角色的视图或一个新派生属性，不需要重跑整个语义层编译，只需刷新视图定义或派生函数，编译内核的“稳定事实”部分因此得以保持低频变更、高频查询的特性。

第 4 章 数据接入与治理

数据接入与治理是语义层"之前"的一段。在原始多源异构数据（银行流水、通话记录、招投标档案、工商登记、轨迹出行、公开 OSINT、举报材料）进入冷层、再被编译成语义层之前，系统必须先回答四个问题：数据以什么格式进来、进来后是否可信、脏值如何被一致地清洗、以及如何把一张新表安全地接进来。本章即描述这一段的实现机制与工程取舍。值得强调的是，这一段的所有模块都遵循"只读观测、不写回断言、不替代人工判断"的底线——适配器只做格式归一，校验只做隔离不毁坏数据，检测器只落诊断不阻断管线，草案只写 `output/drafts/` 绝不触碰 `ontology/`。这种克制不是功能缺口，而是确定性内核对"机器只做计算与提示、绝不做定性结论"这一核心命题的工程兑现。

4.1 五格式适配器与统一 schema

数据接入的第一道关口是格式归一。项目面对的原始文件形态高度发散：银行导出的 CSV、财务系统的 Excel、第三方接口的 JSON、历史归档的 SQLite、以及分析人员早已落地的 Parquet。若让上游每个消费者各自解析这些格式，不仅重复劳动，更会在列名识别、类型推断上产生不一致的口径偏差。`data_ingest.py` 用适配器模式（Adapter Pattern）收敛这一问题：每种格式一个独立适配器类，由统一的 `DataIngestManager`（`data_ingest.py:197`）按文件扩展名分派。

适配器的注册表集中在 `ADAPTERS` 字典（`data_ingest.py:188`），键为扩展名，值为适配器类：`.csv/.tsv` 走 `CSVAdapter`、`.xlsx/.xls` 走 `ExcelAdapter`、`.json` 走 `JSONAdapter`、`.sqlite/.db` 走 `SQLiteAdapter`、`.parquet` 走 `ParquetAdapter`。新增一种格式只需新增一个适配器类并登记进 `ADAPTERS`，主流程 `ingest_directory`（`data_ingest.py:254`）与 `ingest_file`（`data_ingest.py:221`）无需改动——这是"开闭原则"在本项目里少数被明确兑现的地方之一。

五个适配器的职责是单方向的：只读原始文件、做最小必要的列名映射与类型推断，然后产出统一的内部标准 schema。字段映射表 `_COLUMN_MAP`（`data_ingest.py:34`）把"交易日期/date/时间""交易金额/amount/数目""付款方/姓名/name"等常见列名归一到 `{主体, 对方, 金额, 日期, 用途, 来源文件, 来源sheet, 原始行}` 这一套标准列。设计上刻意保留了"原始行"（raw 列），这是后续血缘溯源 `core/row_uri.py` 的物理基础——即便归一出错，原始证据仍在。

这里有一个明确的工程取舍：适配器对待"缺列"采取"容错优先"策略。`_validate`（`data_ingest.py:281`）对银行流水要求 `主体/金额/日期`、通话记录要求 `主体/对端`、中标档案要求 `项目/公司`，但若缺失仅打印警告并保留接入（`data_ingest.py:291`），不中断流程。理由是"正兵（默认角色）可能只上传部分列"，下游语义层会据此降级而非崩溃。与之配套的是类型强制时的"只降级不静默"：金额列含逗号/货币符号会被剥离为数值（`_coerce_types`，`data_ingest.py:51`），无法解析的日期被 `coerce` 为 `NaT`，但损失计数记下 `df.attrs["coerce_lost"]` 随接入记录上报。这呼应了设计原则第 4 条"失败要留痕，不许静默"。

值得注意的是，适配器的类型推断与 4.6 节画像脚本的"raw 只读"是两套互不相让的路径：前者为接入效率做隐式转换，后者为接入前画像刻意 `dtype=str` 不做任何转换（`scripts/profile_table.py:45`），以免金额/日期被 `coerce` 后值类型识别失效。两条路径的并存不是冗余，而是"跑通管线"与"评估数据"两个诉求的分离。

4.2 分区校验与隔离区

格式归一之后、语义层消费之前，每个数据分区（partition）还要经过一道独立的校验闸门，实现于 `core/ingest_validate.py`（REQ-005）。校验的核心抽象是三个数据结构：`IngestPartition`（`core/ingest_validate.py:21`）记录分区元信息（dataset、high_watermark、schema 指纹、行数、内容哈希）；`ValidationError`（`core/ingest_validate.py:33`）携带错误码、分区、详情与严重级别（block/warn）；`QuarantineResult`（`core/ingest_validate.py:42`）汇总隔离结论。

校验维度被收敛为四类，它们覆盖了原始数据最常见的结构性缺陷：

- `DATA_GAP`（数据缺口）：分区表不存在、不可读，或行数为 0。这是最硬的失败，默认 `severity="block"`（`core/ingest_validate.py:79`）。
- `PK_DUPLICATE`（主键重复）：主键列去重后剩余行占比超过阈值（默认 1%，`pk_dup_threshold=0.01`，`core/ingest_validate.py:55`）时报警，防止重复记录污染后续去重与加权。
- `SCHEMA_DRIFT`（schema 漂移）：实际列集与期望列集（`expected_columns`）出现并集差异即报错（`core/ingest_validate.py:91`），用于捕捉上游在不知不觉中改了导出结构。
- `TIME_NON_MONOTONIC`（时间非单调）：对日期类列检查是否存在倒退（`core/ingest_validate.py:115`），这是交易流水被错切、乱序合并的早期信号。

这四类校验码是被显式枚举的有限集合，而非开放的字符串——这意味着校验结果的下游消费（诊断、报表）可以按固定维度聚合，而不必面对不可预期的自由文本。`validate`（`core/ingest_validate.py:52`）逐分区执行上述四项，任何 `severity="block"` 的错误触发隔离动作。`severity` 字段的 `block/warn` 二分也服务于分级响应：`block` 类（表缺失、schema 漂移、主键超阈、时间倒退）必须隔离，`warn` 类（如空分区的 `DATA_GAP` 默认 `warn`，`core/ingest_validate.py:74`）则仅提示、不入隔离——这是“宁可误放提示、不可误糟数据”在严重级别上的细化。

隔离区（`quarantine`）的设计意图是“阻断但不毁坏”。不合格分区不会被删除，而是写入专用表 `partition_quarantined`（`core/ingest_validate.py:174`），通过 `quarantine`（`core/ingest_validate.py:184`）落账，并通过 `is_quarantined`（`core/ingest_validate.py:205`）供下游查询。被隔离的分区从主消费路径退出，但仍可经由诊断回看“为什么被剔”。这是一种典型的 `fail-closed` 思维在存储层的落地：宁可让分区退出计算、也不让带病数据偷偷进入语义层。同时要注意，隔离与第 3 章的“脏值降级”是两层不同的处置：脏值降级在语义层 `CAST` 阶段逐行修复或删除个别字段，而隔离区是针对整个分区的结构性拒收，二者不重叠。

4.3 清洗 op 注册表

跨过隔离区的数据在编译进语义层之前，还要经过清洗（`clean`）/转换（`transform`）处理。`core/clean_ops.py`（REQ-D-004）是这一能力的唯一注册表（`single source of truth`）。强调“唯一”是因为清洗规则一旦散落各处，同一字段在不同表里可能被洗成不同结果，进而破坏确定性内核最看重的“同一输入必得同一输出”。把全部 `op` 收敛到一个注册表，配合 `loader` 的 `fail-closed` 校验（`binding` 引用的 `op` 必须已注册，否则装裁期硬失败），就从结构上杜绝了规则漂移。

注册表的对外接口有四个核心函数。`register_op`（`core/clean_ops.py:35`）注册一个 `op`，结构非法或同名重复注册都会硬失败（REQ-D-004 AC-2，`core/clean_ops.py:59`）；`validate_op`（`core/clean_ops.py:77`）在指定层（`clean/transform`）校验 `op` 是否可用，未知 `op` 或层不匹配直接抛错；`compile_sql_expr`（`core/clean_ops.py:129`）把声明的 `op` 链编译为 SQL 表达式，注入到语义层编译的投影内；`build_clean_context`（`core/clean_ops.py:213`）按 `clean_rules.json`（REQ-D-007）构造清洗上下文，支持 `merge/replace` 两种词表合并语义。

每个 `op` 以 `OpSpec`（`core/clean_ops.py:21`）描述，关键字段是 `impl`（`py` 或 `sql`）、`layer`（`clean/transform/any`）、以及可选的 `sql_template` 与 `param_enum`。`layer="any"` 的 `op` 在 Python 侧与 SQL 投影内均可使用（如 `strip_thousands/strip_currency/cn_date_norm`），因为它们同时提供了 `fn` 与 `sql_template` 双实现；而 `reject_if`、`trim_prefix` 等带参 `op` 仅注册为 `clean` 层，是因为它们的参数（自由文本/正则/换算因子）在 Python 侧处理无 SQL 注入风险，但若进入 SQL 投影层则强制要求 `param_enum` 白名单（`core/clean_ops.py:103`），红线不放宽。

当前已注册的 15 个 op 分别是：`strip_thousands`（千分位逗号剥离）、`strip_currency`（货币符号剥离）、`cn_date_norm`（中文日期归一）、`digits_only`（仅留数字）、`strip_cc`（剥手机国家码）、`strip_paren`（剥括号注释）、`despace`（去空白）、`to_upper`、`to_lower`（大小写归一）、`pad_date`（日期补零）、`reject_if`（条件拒行，`param=contains_mask`）、`trim_prefix` / `trim_suffix`（去前缀/后缀）、`regex_extract`（正则提取）、`unit_convert`（量纲换算）。这套集合覆盖了侦查数据最常见的脏格式，且不依赖任何第三方库，与离线部署目标一致。

需要特别点出与第 3 章的衔接：清洗层不是独立跑一遍再交给语义层，而是在语义层编译时注入。源字段的 transform op 链经 `compile_sql_expr` 编译为 SQL 表达式，注入位置位于 `CAST` 之前——即先 `regexp_replace` 把"¥1,280.50"洗成"1280.50"，再 `TRY_CAST` 成 DOUBLE。这一注入点决定了脏值降级第 1 步（`CAST` 失败）能拿到已经规整过的字符串，而不是原始脏值。第 3 章已展开脏值降级三态细节，本章仅在此确认衔接边界：op 注册表是清洗能力的唯一出处，而 op 链的实际生效在语义层编译之内。

4.4 数据元标准与合规扫描

清洗解决"值长什么样能被解析"，数据元（data element）解决"值应该长什么样"——类型、长度、格式、校验位、敏感度、单位。这是两类不同层级的约束：`clean_ops` 是过程能力，`data_elements` 是验收标准。`core/data_elements.py`（REQ-D-001）是数据元标准注册表，声明落在 `ontology/<pack>/data_elements.json`，平台级共享定义放 `_shared/data_elements.json`（含 `DE_IDCARD/DE_PHONE/DE_AMOUNT/DE_DATE`，以及 `idcard_mod11`、`luhn` 两个校验算法）。

校验算法本身也是注册式的。`CHECKSUM_ALGOS`（`core/data_elements.py:12`）是算法名到校验函数的字典，`register_checksum` 同名重复注册硬失败。`checksum_idcard_mod11`（`core/data_elements.py:26`）实现 GB 11643-1999 公民身份号码校验位（ISO 7064:1983 MOD 11-2），按 17 位加权求和取模 11 对照校验码；`checksum_luhn`（`core/data_elements.py:38`）实现 Luhn 算法（ISO/IEC 7812-1），用于银行卡号/IMEI 等。被声明 `checksum` 的数据元在合规扫描时自动调用对应算法——未知算法在装载期硬失败（fail-closed，REQ-D-001 AC-2），不降级放行。

合规扫描由 `core/compliance.py`（REQ-D-016）执行，签名同检测器 `scan(gateway, ...)`。它只消费已引用数据元（`objects.json` 属性声明 `{"data_element": "DE_X"}`）且已物化的属性，按 `format/checksum/range/enum` 四类检查逐项比对，检查项可单独启停（调用方 `checksum` 参数或 `data_elements.json` 顶层 `compliance_checks` 声明，AC-6）。违规行不以异常中断管线，而是以"对象.属性 + 代理键 + 违规码"落 `run_diagnostic`（`kind=compliance_violation`，`core/compliance.py:162`），违规样本一律经 `_mask_sample` 脱敏。四类违规码分别是 `format_mismatch`、`checksum_failed`、`range_violation`、`enum_unknown`（`core/compliance.py:34`）。

这里有一个清晰的边界：合规扫描是"对官方标准对账"，它只覆盖"已声明数据元的属性"，未引用数据元的列根本不在其视野内（AC-8）。这意味着数据元的覆盖度本身就是治理成熟度的一部分——没有被数据元约束的列，虽然还能进语义层，但失去了机器可读的验收门。这也正是 4.6 节数据元驱动推荐（REQ-D-021）存在的价值：让新表的列尽量挂上数据元，把"野列"收敛到"受约束的列"。

4.5 三类启发式质量扫描

数据元合规是"硬标准对账"，而还有一类问题无法用硬标准覆盖，只能用启发式去嗅探：某列实际是敏感列但没在 `policies.json` 声明遮蔽、某金额列的单位其实和其他表不一致、某张表的流水已经是半年前的旧数据。这类问题由三个同签名 `scan(gateway, ...)` 的检测器承担，分别是 `core/sensitive_scan.py`（REQ-D-018）、`core/unit_scan.py`（REQ-D-020）、`core/data_freshness.py`（REQ-D-019）。

`sensitive_scan` 走两路启发：列名词根（`COLUMN_HINTS`，如 `id_card/手机/卡号`，`core/sensitive_scan.py:29`）与值模式（18 位身份证、11 位手机、16-19 位银行卡，按优先级首中即停、身份证先于泛卡号以控误报，`core/sensitive_scan.py:36`）。值模式命中率需 $\geq$ `hit_ratio`（默认 0.3，`core/sensitive_scan.py:64`）才报，避免偶发命中被放大。已在 `policies.json` 声明遮蔽的属性不再报（去重口径，AC-2）。

`unit_scan` 聚焦金额类数据元（`decimal/integer`）。当同一数据元被多个对象引用、各列中位数跨对象相差 $\geq$ `ratio_threshold`（默认 10000 倍，正好是元与万元的典型量级差，`core/unit_scan.py:44`）时，提示疑似单位混用或金额突增；金额类属性未声明 `unit` 时给 info 级提示（AC-3）。`data_freshness` 按对象 `date` 属性取 `MAX(数据时间)` 与当前日期比对，超 `stale_days`（默认 180 天，`core/data_freshness.py:41`）告警；关键是空时间属性直接跳过、绝不误报为“很旧”（`core/data_freshness.py:67`），且它回答“数据本身多久没更新”，与本体版本新鲜度（`FRESH/STALE`）分开显示、互不混淆。

这三类扫描的统一设计哲学是：只告警不断断。在 `run_all.py` 主数据流里，它们位于语义层编译之后的“质量门”位置（`reference-keypoints.md` 第 11 节），全部落 `run_diagnostic`，返回诊断而不抛异常。为什么不能升级为阻断？因为启发式天然伴随误报：一个账号列里偶有 11 位数字不一定是手机号，金额突增可能是真实的大额交易而非单位混用。把误报变成阻断，等于让噪声拥有否决权，会严重伤害系统的可用性——分析师将频繁被假阳性卡住，久而久之要么绕过、要么无视所有警报。但反过来，“只告警”又带来另一种风险：使用者可能把告警当成机器兜底、误当权威结论。因此三个检测器在输出里反复强调“只提示不定性”“需人工核对”，且下游消费（如画像质量分）对纯启发式扣分标记为“可人工推翻”（`REVIEWABLE_DEDUCTIONS`，`core/ontology_profile.py:225`）。告警是留给人的判断入口，不是机器的判定终章——这正契合核心命题“机器只做计算与提示，绝不做定性结论”。

4.6 列画像与本体草案

接入一张全新外部表时，系统需要先把表“看清楚”，再决定怎么接。这段能力由值类型识别、六层画像、草案组装三块构成，核心约束是“只观察、只建议、不写回”。

值类型识别模块 `core/value_type.py` 是画像的地基。它用两类判定：否定式（否定即排除歧义的可靠信号）`phone/id_card/date_str/amount/account/number`，按特异性降序构成 `ORDER`（`core/value_type.py:33`），顺序即判定优先级——特异性强的排前，避免账号正则吞掉手机号/身份证；肯定式（需确认）`person/org`，由机构名启发词兜底。`analyze_column`（`core/value_type.py:89`）输出类型分布、混装标记（同一列出现 ≥ 2 个归一落点即 `mixed`）与落点建议。混装是这一模块要解决的真实痛点：当 `transaction.from_raw` 里账号与人名混写、`org.raw_name` 里机构名与人名混写时，归一到哪一类不明确，选错就是链断裂。

六层画像在 `core/ontology_profile.py`，由 `OntologyProfiler`（`core/ontology_profile.py:235`）编排，覆盖 L1/L2/L3/L4/L5（L0 见 `core/data_map.py`，见 REQ-P M4 注释 `core/ontology_profile.py:210`）。它消费已物化语义层，逐对象给出空值率、基数、混装、合规违规率、变体数、间类分布等维度，并产出可被人工工资推翻的启发式扣分（如肯定式识别、变体）。需要强调的是，画像与 4.5 的启发式扫描共享“告警非判定”的立场：画像只观察、不写回任何库表，结论一律标“待核实”。

新表接入的画像入口是 CLI `scripts/profile_table.py`。它刻意以 `raw` 模式（`dtype=str`、不做任何类型转换，`scripts/profile_table.py:45`）读外部表，再调 `build_table_profile` 做列画像与候选关联（外部列 `distinct` 与已物化 `obj_*` 可连接属性的 `overlap` $\geq$ 阈值）。随后 `DraftAssembler`（`core/draft_assembler.py`）把画像升级为“诊断 + 生成”：组装出 `objects/links/bindings` 三件草案，每张草案头带 `_draft/_status=待核实/_evidence`，证据链可追溯到具体列与 `overlap` 比。

`DraftAssembler` 有一条被源码扫描固化的红线：全部输出只写 1，绝不写 2（`core/draft_assembler.py:277` 的 `write_drafts`）。其配套函数 `recommend_steps`（`core/draft_assembler.py:303`）零 IO 副作用，只按依赖排序输出

ETL 步骤清单（混装拆分先于绑定、清洗规则先于冷层建表、绑定后必须 reprofile 复检）。这条红线的意义在于：机器生成的草案再像样，也只是"待核实"候选；人工审核复制进 `ontology/<pack>/` 后，还需经 `build_ontology` 的 loader 校验两道闸才生效。草案与生效声明之间的人为闸门，正是"声明是数据、实现是代码"这一设计原则在新表接入场景的具体兑现。

4.7 新数据源接入的六步路径

把前述各模块串起来，新数据源接入形成一条固定六步路径（见 `reference-keypoints.md` 第 16 节），其最大特色：全程不改任何检测器的 Python 代码。这并非偶然便利，而是"声明是数据、实现是代码"与"写路径唯一/检测不改"等设计目标的直接兑现。

第一步，`profile_table.py` 对列做画像与值类型识别，产出列画像、候选关联、以及 4.4 提到的数据元驱动推荐（`core/de_recommend.py`，REQ-D-021）。推荐器有两条置信阈值：`CONFIRM_THRESHOLD=0.70`（`core/de_recommend.py:25`，命中率低于此值不推荐）与 `HIGH_THRESHOLD=0.90`（`core/de_recommend.py:26`，达此值且为强信号才标 high，否则 medium 需人工确认）。它永不动写 `objects.json`、低置信一律标 `needs_confirmation`、混装复合列只给拆分提示——红线与 DraftAssembler 同源。

第二步，`DraftAssembler` 生成 `objects/links/bindings` 三份草案，只落 `output/drafts/`，绝不碰 `ontology/`。第三步，分析师审阅草案，确认对象名、pk、name_property、候选关联后，手工复制到 `ontology/<pack>/`。第四步，若新表需要新计算能力，在 `functions.json` 声明（Python 实现还需注册 `FUNCTION_IMPLS`），并在 `rules.json` 写 `rule_text` 与参数。第五步，在 `policies.json` 补策略声明——这一步极易被遗漏，但遗漏会在运行时被 fail-closed 拒绝（未声明即拒绝，呼应设计原则第 5 条），所以必须显式补齐。第六步，`validate_ontology.py --strict` 做严格校验，通过后 `build_ontology.py` 编译生效。

六步路径的精妙处在于它的"封闭性"：检测器（`compliance/sensitive_scan/unit_scan/data_freshness`）的能力边界由数据元声明、属性遮蔽声明、值模式启发式决定，而这些都是声明式配置。接入一张新表时，分析师改的是 `ontology/` 下的 JSON 与 `clean_rules.json` 词表，而非去改 `core/compliance.py` 的正则或 `core/sensitive_scan.py` 的 `COLUMN_HINTS`。当一份新数据需要新校验逻辑时，正确做法是把它表达为新的数据元声明（挂 `format/checksum/range`）或新的属性遮蔽策略，而不是 fork 检测器。这种"能力通过声明扩展、实现保持稳定"的纪律，保证了内核在持续接入新数据来源时，不会因为检测器各自膨胀而丧失整体一致性与可审计性——这也是第 8 章"已知缺陷与技术债"中需要持续守护的边界之一。从 `reqd_case` 观察，即便接入已经过精心构造的脏数据路径（缺对端列的 `call_old`、整列降级的 `cp_whole`、拆分路径的 `cp_split`），只要声明层面的列映射与降级策略写清，编译期与运行期就能给出确定且可回放的处置，而不必回头改一行检测器代码。这条路径的可重复性，正是六步接入设计价值的最终体现。

第 5 章 规则引擎与 Function 计算层

本章是全书的技术核心章。第 3 章已经把"数据是什么、从哪来"编译成 `obj_*` / `lnk_*` 语义层，第 4 章解决了"数据怎么进来、质量怎么把关"。本章回答的问题是：在语义层之上，机器如何把"侦查判据"翻译成可复现、可回放、可追责的确定性计算。围绕这一目标的工程取舍是：判据（写什么）与执行（怎么跑）与计算（算什么）必须彼此解耦，使得"加一条规则"成为一次声明式编辑，而不是一次 Python 代码改动。本章不重复术语、禁令、架构与数据接入，只聚焦检测逻辑的组织方式、执行机制与边界。

5.1 三层分离：判据、执行、计算

本章立论的核心，是把"检测到什么"这件事拆成三个互不污染的层：判据层、执行层、计算层。判据层是 `ontology/<pack>/rules.json`（规则手册，Rulebook），它只描述"在什么维度、依据什么自然语言判据、挂靠哪个 Function、传什么参数"——`function` 字段加 `params` 是唯一机器挂钩，机器不解析 `rule_text` 自然语言，只认只读 Function（`core/rules.py:1-15` 的模块头注释明确"本模块不解析自然语言"）。执行层是 `core/rules.py`，它负责按阶段过滤规则、解析阈值、调用 Function、做命中判定、抑制重叠、归类零命中，是一个与具体业务判据无关的通用驱动器。计算层是 `ontology/<pack>/functions.json` 加 `core/functions.py`，前者声明每个计算单元的输入、参数白名单与 SQL 模板，后者提供只读执行器 `FunctionExecutor` 与 Python 实现的注册表 `FUNCTION_IMPLS`。

这一分离的直接收益是：新增一条检测规则，分析师只需在 `rules.json` 写下 `rule_text` 与 `function / params`，若已有可复用 Function 则完全不改 Python；即便需要新计算能力，也只是声明 `functions.json` 并在 `FUNCTION_IMPLS` 注册一个函数（装载期校验 `impl_ref` 存在，未知即硬失败）。判据的增删改因此对代码库零侵入。

它也与第 3 章的"八段声明模型"在精神上同构：`rules.json` 属规则手册段、`functions.json` 属能力目录段，二者都只是声明数据；真正的执行器（`core/rules.py`）与计算注册表（`core/functions.py`）是通用 Python 代码，不随业务变。这意味着"加规则"与"改内核"是两条正交的变更流——内核稳定、声明易变，这正符合第 1 章六条原则之二的"声明是数据、实现是代码"。正因如此，第 16 节所述"新数据源接入六步路径"全程不要求改检测器 Python 代码：接表改 `bindings`、加能力改 `functions.json`、写判据改 `rules.json`、补策略改 `policies.json` 即可。

这也就解释了为什么项目里没有 `detectors` 目录：调查规则在架构上不是"每个规则一个检测器类"，而是声明数据经通用引擎驱动。需要澄清的是，`core/` 模块地图的 H 组确实名为"检测器"，但它仅含 4 个同签名 `scan(gateway, ...)` 的数据治理扫描器——`compliance.py`（数据元合规）、`sensitive_scan.py`（敏感列）、`unit_scan.py`（单位口径）、`data_freshness.py`（数据新鲜度），归属 REQ-D 系列，是数据质量探测器而非侦查规则探测器（`reference-keypoints.md` 第 4 节 H 组）。侦查侧的"detector"已被 `rules.json` + `functions.py` 的声明式范式吸收，因此没有独立的 `detectors` 目录是设计结果，不是实现缺失。

三层分离带来的第二重收益是阶段隔离与增量重算。`run_rules(store, stage="xu_shi", pack, rule_ids, health, base_dir)`（`core/rules.py:141`）第一道过滤就是 `stage`：只跑当前技能阶段对应的规则，R1 - R5 归 `xu_shi`、R6 归 `qi_zheng`，互不污染。`rule_ids` 非空的入口则只重算指定规则（`core/rules.py:168`），这正是第 6 章人审闭环（REQ-016）"只重算 `affected_rules` 的物理基础——改一条判据、只跑这一条、不动其余，确定性来自"输入相同的声明 → 输出相同的执行"，与 Python 代码改动彻底解耦。判据层出错时定位也变简单：`finding` 自带 `rule_id / rule_text`，审计者能直接回到 `rules.json` 的原文，而不必在代码里翻 detector 实现。

5.2 R1-R6 规则详解

现有包 `default`（张卫国受贿/串标案）共挂载六条检测规则 R1 - R6，全部落在 `xu_shi`（虚实）阶段，唯独 R6 落在 `qi_zheng`（奇正）阶段，承担奇兵拓线职责。`rules.json` 中每条规则都带有 `dimension`（维度）、`jian_types`（间类）、`assumption`（假设编号）、`function`、`params`、`hit_when` 等字段；其中 `hit_when` 决定命中判定方式——`rows_nonempty` 表示 Function 返回非空即命中，`result_hit` 表示读取 `result.hit` 布尔（见 `core/rules.py:26` 的 `_evaluate_single`）。逐条说明如下。

从分布看，六条规则覆盖五个维度中的资金(3条)、通讯、行为、关系、时间各1条，资金维度权重偏高（R1/R2/R6 三条都围绕整数金额，其中 R6 时间维度依附资金判据），这是 `reference-keypoints.md` 第 18 节已记录的已知风险“规则偏少、资金占3条”。所有规则的 `assumption` 字段指向庙算假设 H1 - H4，构成“规则命中 → 假设佐证”的回灌链路：R1→H1（收受财物）、R3→H3（密切私下关系）、R5→H2（利益关联）、R2/R6→H4（第三方过桥）。值得注意的是 `excludes` 字段（R1 排除 R2、反之亦然）与 `exclusive_group` 是两套互补机制：前者是声明层意图声明，后者是执行层实际消解逻辑。逐条的事实细节如下，每一条都附带其降级路径，因为降级路径决定了“这条规则在数据源不足时如何优雅退场”而不是崩溃。

R1「季度末整数现金存入」挂靠 `quarter_end_integer_deposits`，维度资金、间类生间、假设 H1，组内 `primary_rule=true`。判据为金额对 `round_unit=10000` 取模为 0 且 `to_raw='现金存入'`（参数 `cash_summary_tokens`），日期距季末边界 `≤ quarter_end_window_days=15` 天，按季度聚合。它与 R2 同属 `exclusive_group=integer_amount`，承担组内 `primary` 角色。降级路径：当 `obj_transaction` 缺失或列不符时，由 Function 层抛 `CatalogException` / `BinderException` 触发结构降级为零命中并标 `degraded`，不崩管线（`core/functions.py:641`）。

R2「整数转账聚合（第三方过桥）」挂靠 `integer_transfer_aggregates`，维度资金、间类反间、假设 H4。判据为 `amount % 10000 = 0`，按 `from_raw→to_raw` 聚合单向大额链条，`subject_column=from_raw`。它与 R1 互斥：`exclusive_group=integer_amount` 且 `primary_rule=false`，`overlap_resolution=drop_if_primary_hit`——当 R1（`primary`）命中时 R2 被抑制（详见 5.7）。降级路径同 R1。

R3「招投标公示期通话频次突增」挂靠 `call_frequency_spike`，维度通讯、间类生间、假设 H3，`hit_when="result_hit"`。判据为头部对端通话数 $\geq$ 其余对端中位数 $\times 2$ 。其降级路径最值得关注：当某主体只有单一通话对端、无法计算常态中位数时，函数自动降级为绝对阈值 `absolute_threshold=30` 并标 `is_degraded=true`（`core/functions.py:176-185`），且 `call_pair_coverage` 可独立诊断“是否仍运行在不可靠的绝对阈值模式”。

R4「二人公示期轨迹同框」挂靠 `co_located_pairs`，维度行为、间类生间。它直接 `SELECT lnk_co_located`（不同主体同地点 ± 1 天），`subject_column=person_1`，无参数。降级路径：语义链接未编译或为空时结构降级为零命中。

R5「工商登记利益关联」挂靠 `org_interest_links`，维度关系、间类因间、假设 H2。`params` 为空，判据由 `case_knowledge.json` 驱动：遍历 `obj_org` 的 `legal_rep / relation` 是否命中知识包中的 `subject_aliases` 或未过期 `relation_assertions`（人名只合法存放于 `case_knowledge.json`，函数与规则代码中不写任何人名，`core/functions.py:477`）。降级路径有两个：知识包缺失→`config_missing` 零命中；`obj_org` 缺失→结构降级。

R6「中标-资金时间窗碰撞」挂靠 `time_window_collision`，维度时间、间类反间、假设 H4，落在 `qi_zheng` 阶段。判据为 `lnk_time_window JOIN obj_bid_project`：金额 `%10000=0` 且 `owner_raw NOT LIKE '%公司%'`（排除对公，只留个人），按 `offset_days` 排序，`subject_column=资金主体`。降级路径同其余 SQL 实现。

需要特别警惕一处术语陷阱：代码注释里出现的 R5/R9/R13 等是 REQ 需求编号，不是检测规则编号；本系统的检测规则只有 R1 - R6（`reference-keypoints.md` 第 6 节警告）。写作与审计时不得混淆。

5.3 11 个 Function 与两类实现

计算层共 11 个 Function (Function)，分两类实现：4 个 SQL 实现、7 个 Python 实现。这条分界不是随意的，而是由“计算能否用单条只读 SQL 表述”决定——能则 SQL（强白名单、易审计），不能（需要多步聚合、跨源印证、归并逻辑）则 Python（注册为可信代码）。

4 个 SQL 实现为 `quarter_end_integer_deposits`、`integer_transfer_aggregates`、`co_located_pairs`、`time_window_collision`。它们在 `functions.json` 中以 `impl:"sql"` 声明，并携带 `sql` 模板。SQL 实现受双重白名单约束：首先 `_FORBIDDEN_SQL` 正则（`core/functions.py:24`）命中 `INSERT/UPDATE/DELETE/DROP/CREATE/ALTER/ATTACH/DETACH/COPY/TRUNCATE/GRANT/REVOKE` 即拒；其次 `_assert_readonly()`（`core/functions.py:30`）要求语句首词必须是 `SELECT` 或 `WITH`。参数经 `render_sql_template()` 渲染为类型化字面量，占位符 `{{param}}` 与 `parameters` 必须一一对应——多声明或少使用都硬失败（`core/functions.py:112-121`），从根上消除参数注入。

7 个 Python 实现为 `call_frequency_spike`、`call_pair_coverage`、`jian_cross_level`、`tipoff_cross_reference`、`org_interest_links`、`overpass_two_hop`，以及 `location_colocated`（实现独立在 `core/geo.py`，仅在本模块末尾注册进 `FUNCTION_IMPLS`，`core/functions.py:668-670`）。Python 实现走注册机制：`FUNCTION_IMPLS` 字典由 `@register_function(name)` 装饰器填充（`core/functions.py:139`）；`FunctionExecutor.invoke()` 通过 `inspect.signature` 兼容新旧签名——新签名 `fn(store, params, ctx)` 接收 `RuntimeContext`（可 `ctx.table()` / `ctx.link()` 取只读表名），旧签名 `fn(store, params)` 向后兼容（`core/functions.py:586`）。所有 Python 实现通过 `ReadOnlyStore` 代理访问数据（`core/functions.py:548`），且调用前由 `PolicyEngine` 对 `inputs` 中每个 `obj_*` / `lnk_*` 做 fail-closed 策略检查（`core/functions.py:609`）。

这套两类实现的划分对可审计性有实质意义。SQL 实现从声明到执行全程可静态审阅：`functions.json` 里的 `sql` 模板是“白纸黑字”，配合 `_FORBIDDEN_SQL` 与首词校验，审计者不必运行即可确认它只读、不拼接外部文本。Python 实现则相反——逻辑写在代码里，无法靠模板静态保证安全，因此用更重的护栏兜底：一是 `FUNCTION_IMPLS` 注册表把“可用函数”收敛为白名单，装载期校验 `functions.json` 的 `impl_ref` 存在（未知即硬失败，杜绝声明了一个不存在的实现），二是 `ReadOnlyStore` 在对象层屏蔽 `execute / conn` 等写入口（`core/runtime_context.py`），即便实现代码误写写操作也会在代理层被挡下。两条轨共同的出口是 `invoke()` 返回的字典：SQL 实现塞 `rows`、Python 实现塞 `result`，这一路径差异由 `_evaluate_single` 统一消化（`core/rules.py:37-49`）——也是 REQ-G-022 修复的历史坑：原先只读 `out["rows"]` 导致 R5 这类 `result` 路径的 py 实现永不匹配。

5.4 只读红线与防注入

Function 层是写路径之外的只读计算域，其红线目标是：任何声明的计算都无法修改数据、无法逃逸到未声明的语义表、无法注入任意文本。约束由三层叠加。

第一层是 SQL 关键字黑名单与首词校验，已在 5.3 说明（`_FORBIDDEN_SQL` + `_assert_readonly`）。第二层是 `{{param}}` 双向核对：模板里出现的占位符集合与 `parameters` 声明集合必须相等，缺声明或多声明都直接抛异常（`core/functions.py:114-121`）。这一设计意味着“模板少写一个参数”或“声明里多一个无用参数”都会在执行前失败，而不是把错位参数静默带入查询。第三层是参数值本身的类型与白名单校验，集中在 `check_param_value()`（`core/functions.py:75`）：`integer` 拒绝 `bool`（因 Python 中 `bool` 是 `int` 子类，必须显式排掉）、`decimal` 接受 `int/float` 同样排除 `bool`、`boolean` 严格用 `isinstance` 判定、`date` 必须是 `YYYY-MM-DD` 正则匹配；最关键的是 `string` 类型必须声明 17 白名单——没有白名单或取值不在白名单内一律拒绝，自由文本不接受（`core/functions.py:90-96`）。例如 `quarter_end_integer_deposits` 的 `cash_summary_tokens` 与

`time_window_collision` 的 `exclude_org_suffix` 都只能取 `["现金存入"]`、`["公司"]` 之类枚举，从声明侧消除把任意字符串拼接进 SQL 的可能。

此外还有结构降级护栏：当 `py/sql` 函数抛 `CatalogException` / `BinderException` 且异常信息引用了 `obj_*` / `lnk_*` 语义层（`_is_structural_degrade`，`core/functions.py:47`），说明数据源未接入或 `schema` 不符，此时返回零命中并标 `degraded` 留痕（`core/functions.py:641-657`），而非让单条规则异常拖垮整条管线。这是“失败要留痕、不许静默”原则在计算层的落地。

逐条对照这些护栏，可以看清它们各自封堵的具体攻击面与出错模式。`_FORBIDDEN_SQL` 挡的是“声明的 Function 偷偷写了数据”——例如某分析师想用 Function 顺手建临时表，正则直接拒绝，因为 Function 不是写路径（`core/action_executor.py` 才是唯一写入口）。首词 `SELECT` / `WITH` 校验则挡住更隐蔽的写法，比如以注释或 CTE 名义夹带 `DELETE` 的变体。`{{param}}` 双向核对挡的是“参数漂移”——模板改了占位符却忘了更新 `parameters`，或反过来，两种都会让渲染出的 SQL 语义错位。`string` 必须 `enum` 这一条是 SQL 注入的最后防线：即便有人绕过模板直传参数，`check_param_value` 仍强制取值落在白名单，自由文本（如含单引号的恶意串）一律进不来。而 `integer` 拒绝 `bool` 是为防 Python 语义陷阱——`True` 在 DuckDB 里会被当成 `1`，若阈值参数被误传布尔会悄然改变判据量级。这些约束共同构成“声明即受信、但受信不等于不受限”的边界。

5.5 阈值策略与自适应

阈值是检测规则的灵敏度旋钮，其策略对象由 `core/threshold.py`（REQ-027）承载。核心入口 `resolve_rule_params(store, rule_id, params, pack)` 根据 `thresholds.json` 的声明，把 `rule.params` 副本中 `bound_param` 指向的阈值替换为计算值，并返回 `(params_copy, method, threshold_value, is_degraded)`。计算保持只读，绝不写回 `rule.params` 原值（`core/threshold.py:5-10`）。

它支持两种 `method`：`absolute`（绝对阈值）与 `relative_median`（分位数自适应）。`relative_median` 用于 R3 这类“相对常态”判据——从 `obj_call` 拉取频次样本，样本数 $\geq$ `min_samples`（默认 20）时取中位数 $\times$ `multiplier`（默认 2.0）作为阈值，并向上取整为整数次数（`core/threshold.py:139-163`）。这把“突发”定义为相对自身常态的倍数，比硬编码常数更抗数据分布漂移。但自适应必须戴两道枷锁：其一是 `bounded_by` 夹紧（`_bounded`，`core/threshold.py:79`），无论自适应算出的阈值多高多低，都被限制在 `thresholds.json` 声明的 `[min, max]` 区间内，防止极端样本把判据推到荒谬位置；其二是样本不足时绝不硬算——样本数 $<$ `min_samples` 时回退到 `fallback` 值（或无声明时回落 30）并设 `is_degraded=true`（`core/threshold.py:164-171`），显式告知“本次阈值是退化的，结论需谨慎”。无论哪条路径，`is_degraded` 都会随 `finding` 透传给产物（`core/rules.py:207`），与 R3 函数自身的绝对阈值降级共同构成双重可审计标记。

自适应阈值的工程实现还带一个常被忽视的约束：确定性。`resolve_rule_params` 在 `relative_median` 分支内固定使用 `seed=20260501` 的 `random.Random`（`core/threshold.py:143-145`），即便当前样本路径不依赖随机，也显式消费一次随机数以保证“同一份输入 $\rightarrow$ 同一份阈值”在固定 `seed` 下可复现（REQ-027 AC4）。这与全书“确定性优先于智能”的原则一致——自适应不是引入不确定性，而是在确定性框架内选取阈值。另一个值得说明的现状是：`_collect_samples`（`core/threshold.py:90`）目前只对 R3 实现了样本拉取逻辑，其余规则尚返回空列表（注释明确“以后扩展时可按 `rule` $\rightarrow$ `sample SQL` 映射”）；这意味着除 R3 外，阈值目前仍走 `absolute` 或 `absolute_hardcoded` 路径，自适应能力是按需启用的而非全局默认。设计上这留了扩展点，但当前并不假装“所有规则都已自适应”。

5.6 零命中四分类

调查系统最危险的失败，不是漏报，而是把“数据根本没接进来”静默呈现为“没有异常”。`core/rules.py:120` 的 `_classify_zero()` 与 REQ-G-002 把零命中拆成四个语义不同的类别，目的是强制区分“没查”与“查了没中”。

- `data_absent` (info)：数据本来就没有——语义表缺失、列不符导致结构降级，或输入表 0 行。这不是"干净"，是"没数据"。
- `config_missing` (warning)：配置或知识包为空导致函数无法计算（如 R5 无 `case_knowledge.json`）。
- `empty_result_suspect` (warning)：数据齐全却零命中，可疑——可能匹配逻辑失效，需要人工确认。
- `clean_scan` (info)：确属干净。必须由分析师在 1 显式声明 2（当前六条规则均未声明），系统不许自行把零命中解释为"排除"。

分类逻辑（`core/rules.py:120-138`）的优先级值得细读：若函数已标 `degraded` 则归 `data_absent`（结构降级已由 Function 层记 warning）；其次若函数自报 `config_missing` 则归 `config_missing`；若扫描到的输入语义表总行数 `scan_rows==0` 则归 `data_absent`；只有当有输入、未声明 `clean`、且函数正常返回空时，才归 `empty_result_suspect`。`scan_rows` 由 `_scan_rows()`（`core/rules.py:102`）统计 Function 声明消费的 `obj_*` / `lnk_*` 总行数，表不存在计 0。

这一步的关键价值在于：一条规则零命中时，系统不再裸 `continue`，而是向 `run_diagnostic` 落一条带 `zero_type` 的诊断对象（`core/rules.py:186-191`），且诊断与 findings 物理分离（原则四）。于是审计者能立刻区分——"R4 零命中是因为 `lnk_co_located` 表压根没编译"（`data_absent`，属接入问题）与"R4 编译了但确实没有异人同地记录"（`empty_result_suspect`，属可疑需复核）。前者若被误读为后者，会让调查在错误的安全感中错过关键缺口。

最后一类 `clean_scan` 的"显式声明"要求是这套分类里最容易被挑战、却最必须坚持的一环。设计上我们选择 fail-closed：系统绝不自行把零命中解释成"排除了嫌疑"，`zero_is_clean` 必须由分析师在 `rules.json` 里写 `true` 才生效（`core/rules.py:136`）。理由是"没查到"在侦查语境下永远首先是"证据缺口"而非"清白证明"，让机器默认下排除结论，既越过了第三条禁令的边界，也会在复核时误导后人。因此当前 `default` 包六条规则无一声明 `zero_is_clean`，它们任何一条零命中都会落为 `empty_result_suspect` (warning) 或 `data_absent` (info)，把"是否需要补数据/查检测器是否失效"的决策显式留给人类。这四类标签与 `run_health` 的诊断种类（`reference-keypoints.md` 第 10 节）一一对应，最终汇入 `run_diagnostic` 与 findings 物理分离的统一出口。

5.7 互斥组与重叠消解

当两条规则本质刻画同一类现象、只是口径互补时，同时命中会产生冗余甚至矛盾的线索。`core/rules.py:52` 的 `_suppress_overlaps()`（REQ-025）负责在末段消解这类重叠。机制是 `exclusive_group` 声明：R1 与 R2 同属 `integer_amount` 组，但 R1 `primary_rule=true`、R2 `primary_rule=false`，且两者 `overlap_resolution="drop_if_primary_hit"`。

执行流程分三步：先收集所有 `exclusive_group` 中已被 `primary` 命中的组号集合；再逐条检查 finding——若其所在组已被 `primary` 命中、自身非 `primary`、且策略为 `drop_if_primary_hit`，则把它移入 `suppressed_log` 并从主列表移除，同时在该 finding 上挂 `suppressed_reason` 与指向 `primary` 的 `suppressed_by_rule`；最后把完整 `suppressed_log` 附到每条保留的 finding 上（`core/rules.py:52-99`），保证审计链完整、产物可还原"谁抑制了谁"。语义上，现金存入（R1）与整数对公转账（R2）都围绕"整数金额"，但前者聚焦个人现金走账、后者聚焦第三方过桥，组内以 R1 为优先、R2 仅在 R1 未命中时才进入主列表，避免同一笔整数资金被两条规则重复标记为两类候选。这是声明式消解而非代码分支，新增互斥关系只需在 `rules.json` 配 `exclusive_group` / `primary_rule` / `overlap_resolution` 三字段。

一个容易被误读的细节是：被抑制的 R2 并非被彻底丢弃。`_suppress_overlaps` 把它的 `suppressed_log` 条目（含 `suppressed_by_rule`、`reason`、原候选出处 / `source_rows`）挂到保留的 finding 上（`core/rules.py:80-98`），实现"主列表干净、但消解关系可审计还原"。这是 REQ-025 的核心要求——重叠消解不能让信息消失，只能让"主视图"不重复，审计者随时能从 `suppressed_log` 看到"R2 当时也命中、只是被 R1 压下了"。当前 `overlap_resolution` 只实

现了 `drop_if_primary_hit` 一种策略；若未来需要“保留低优先级但降级标注”等更细粒度消解，扩展点仍在 `rules.json` 的该字段而非 Python 分支。

5.8 庙算：五间 × 五维覆盖模型

“庙算”是侦查研判前的沙盘推演，对应代码 `core/hypotheses.py` 的 `MiaoSuan(pack)`。它的职责不是下结论，而是把虚实扫描产出的 findings 映射为候选假设，并量化“覆盖是否完整、对抗痕迹是否遗漏”——这是反遗漏（不漏掉该查的方向）的工程抓手。

五间（间类）由 `jians.json` 声明式定义：`因间`（org,bid_project,weight 3）、`内间`（tipoff,weight 5）、`反间`（transaction,weight 2）、`死间`（osint_article,org,weight 4）、`生间`（call,trackpoint,weight 1）（`ontology/default/jians.json`）。注意 `source_object_types` 决定哪些语义对象属于该间类，名称可从声明增删，但间类的语义归属由声明驱动而非硬编码。五维（维度）由 `dimensions.json` 定义：资金(transaction)、通讯(call)、行为(trackpoint)、关系(person/org)、时间(transaction/call/trackpoint)。

五间模型最关键的工程概念是交叉升格（cross-level escalation）：单源证据只是“观察”，两个独立源印证升格为“线索”，三个及以上源升格为“可立案依据候选”（`core/functions.py:360-362`、`core/lineage.py:157`）。其映射 `min_independent_sources` 为 1/2/3 这一关系硬编码不可配置，只有等级名称（观察/线索/可立案依据候选）可从 `cross_levels` 配置（`jians.json` 的红线注释与 `reference-keypoints.md` 第 7 节均强调）。这样设计的用意是：升格逻辑是方法论常量，不应因配置漂移而改变；而名称可本地化。更重要的是，即便达到“三源”，系统也只标“可立案依据候选”——机器绝不置“已立案”，与第三条禁令（不下定性结论）严格一致。

`MiaoSuan` 的覆盖模型提供四类可量化检查：`dimension_coverage()` 双轨口径——`declared`（假设声明了哪些维度，理论覆盖）与 `empirical`（扫描实际命中的维度，实证覆盖），且两者独立报警（REQ-G-009/024），因为“假设写了维度但扫描无命中”属“想到了没查到”，与“压根没想到”是不同补救动作；`jian_coverage()` 检查每个间类是否被假设引用，缺则警告“对抗痕迹未覆盖”；`conflict_check()` 检测两条假设是否引用同一笔 `source_rows`（提示需合并或区分）；`enumerate_space()` 用笛卡尔积展开枚举候补池，命中 `ENUM_BEHAVIOR_MAP` 的有检测器支撑可转正，其余入 `backlog` 待人工注入，且枚举空间永不闭合（传自定义 space 即可扩维）。假设本身受 `MAX_HYPOTHESES=5` 上限与“知己栏强制非空”约束（`core/hypotheses.py:163-165`），确保沙盘可控、可追溯。

把庙算放回五技能（skill）流水线能更清楚它的位置：技能链是 `miaosuan`（庙算）→ `zhi_ji_zhi_bi`（知己）→ `xu_shi`（虚实）→ `qi_zheng`（奇正）→ `yong_jian`（用间）（`skills/registry_bootstrap.py:245`）。庙算处于最前，负责“开打前先想清楚覆盖哪几个方向、漏了哪个维度、假设之间有没有抢同一笔证据”。`jian_cross_level` 这个 Function（`core/functions.py:332`）则在案件级独立计算五间交叉等级——它遍历 `obj_*` / `lnk_*` 中声明了 `jian` 的表，非空即算“该间类有数据源命中”，再用 `count_independent()`（`core/functions.py:276`，按 `source_independence.related_pairs` 并查集合并同源）算出独立源数，最终落到 1/2/3 的硬编码映射。需要强调交叉升格的保守性：单源只是“观察”、双源才“线索”、三源也仅是“可立案依据候选”——它把“证据的强度”显式分级，但始终保持“机器只提示、不置定性结论”的边界。未建模的数据源在 `jian_cross_level` 里会被诚实标为“缺口”而非充数（`core/functions.py:350`），这也呼应了零命中四分类“不得用缺口冒充干净”的同一纪律。

5.9 血缘去重与优先级排序

多条技能（虚实/Q2 过桥/Q3 通话等）可能从不同角度产出指向同一事实的线索，全量堆给正兵会造成信息过载。`core/lineage.py` 基于“血缘”做去重合并与优先级排序，使工作台只看最值得查的线索。

去重依据是血缘三要素（`core/lineage.py:9-17`）：① 数据溯源重叠度——两条线索 `source_rows` 的 Jaccard 相似度 $\geq$ `threshold`（默认 0.5）；② 间类归属——合并后 `jian_types` 取并集；③ 假设链——同 `assumption_chain`

推导属同一逻辑链。`dedupe_and_merge(clues, threshold=0.5)` (`core/lineage.py:52`) 用并查集把同源线索归组，每组取字段并集、拼接 `title`、保留 `merge_log` 记录"由哪些原始线索合成"，返回数量 $\leq$ 输入。合并后若 `jian_types` 增多，会触发 5.8 所述的交叉升格——这正是"多源印证升格"在血缘层的落地：单源线索合并不再是单源，可能从"观察"升为"线索"。

优先级排序由 `prioritize_clues()` (`core/lineage.py:210`) 完成，按 `confidence` × 间类覆盖 × 数据强度 三维打分降序：置信度取假设链中最高者（默认 $H1=0.9$ 、其余 $=0.7$ ），间类覆盖取命中间类最大权重 / 归一化基数，数据强度取 `min(1.0, 溯源行数 / 10)`。权重从 4/5 声明读取（R7，不再硬编码），每条线索被打上 `priority_score / score_basis / cross_level` 供操作台展示。这样既让"高密级内间 + 多源印证 + 数据扎实"的线索排在最前，又保证排序逻辑可声明、可解释、可审计。至此，从判据声明、执行驱动、只读计算、阈值自适应、零命中留痕、重叠消解、庙算覆盖到血缘去重，整条检测链路在"确定性优先、失败留痕、写路径唯一"的约束下闭环。

去重阈值 `threshold=0.5` 是整个合并的灵敏旋钮：它要求两条线索的溯源行 Jaccard 相似度达一半以上才并组，过低会把"只是恰好引用了同一张表不同行"的线索误合并，过高则留不下去重效果。工程上它是 `dedupe_and_merge` 的入参（`core/lineage.py:52`），调用方（`run_all.py` 主线）传 0.5，未硬编进函数体，留了调参空间。`same_assumption` 与 `jian_intersect` 的"同假设链且间类有交集"作为补充合并条件（`core/lineage.py:93-95`），则兜住了"溯源行不完全重叠但逻辑同链"的情形——例如 R3（生间）与 R4（生间）都指向"二人私下关系"，即便引用的是通话表与轨迹表不同行，仍应合并升格。最后，线索处置状态通过 `save_statuses / load_statuses`（`core/lineage.py:310/344`）以 `INSERT ... ON CONFLICT DO UPDATE` 幂等落 L2 DuckDB，使正兵跨会话仍能恢复上次进度，审计链则保留在内存 `LineageClue.audit_log` 而非状态表，避免处置状态与审计职责混淆。

第 6 章 线索生命周期、处置与写路径

本章聚焦系统的写入侧（write path）：从规则引擎产出的 finding 转换为线索（clue）之后，如何被正兵（人类处置者）跟踪、处置、固证、立案，并最终闭环回到语义层与案例库。与第 1 - 5 章不同，本章不重复术语、架构与规则计算的细节，而是把笔力集中在"谁有资格写、写之前经过哪些校验、写之后如何对外确认与对账、异常与暂缓线索如何被隔离"这一条纵贯线上。核心命题是：在确定性侦查内核里，写操作不是功能的附属动作，而是受治理约束的第一类公民。所有写路径收敛到唯一的 `ActionExecutor`，所有状态迁移受状态机与角色边界双重约束，所有对外回写经发件箱（outbox）与对账（reconcile）保证不丢、不重、不假装在途。

6.1 线索模型与状态机

线索模型以 `LineageClue`（`core/registry.py:147`）为内存标准单元，承载假设链（`assumption_chain`）、数据溯源（`source_rows`）、间类归属（`jian_types`）与处置状态（`status`）。处置状态由 `ClueStatus`（`core/registry.py:38`）以五态常量表达：待查（`PENDING`）、查证中（`VERIFYING`）、已排除（`EXCLUDED`）、已固证（`CONFIRMED`）、已立案（`FILED`）。状态不是自由字符串，而是受状态机约束的有限集合，任何变更都必须经由 `set_status` / `set_filed` 两个方法（`core/registry.py:180`、`:210`），禁止直接赋值 `.status`，以保证审计链（`audit_log`）完整。这一设计把"状态从哪来、经过谁、为什么变"固化进数据结构，而不是散落在各处的过程代码里。

状态机的核心是一张合法迁移表 `ClueStatusMachine._TRANSITIONS`（`core/registry.py:73`），它定义了五态之间的可达关系。待查可前进到查证中、已排除或已固证；查证中可回退到待查，也可走向已排除或已固证；已排除并非不可逆，可因新证据重开回待查，这是侦查中"排除结论被新线索推翻"的真实需要；已固证可落到已排除或已立案；已立案是封闭终态，不再外溢。迁移表并非凭空写死在 Python 里——正式包从 `states.json` 读取（见 `ClueStatusMachine.transitions_for`，`core/registry.py:82`），默认表只是回落值，确保状态语义随案件包声明而演进。

当前状态	可迁移到	说明
待查	查证中 / 已排除 / 已固证	初始态，正兵接手或机器初判
查证中	待查 / 已排除 / 已固证	核查中可回退或升级
已排除	待查	排除可因新证据重开
已固证	已排除 / 已立案	稳定证据后走向终局
已立案	（无）	受控终态，仅人类可置位

五态中最关键的是"已立案"的专属地位。系统通过两条互锁的防线把它锁死在人类手里：其一，`core/access.py:37` 定义 `HUMAN_ONLY_STATUSES = frozenset({"已立案"})`，并在 `access.can_transition`（`core/access.py:140`）中硬性拦截非人类角色向该态的迁移；其二，`LineageClue.set_filed`（`core/registry.py:210`）要求当前状态必须处于已固证或已立案，且必须携带法定依据（`legal_basis`），`set_status` 则直接拒绝把目标设为已立案（`core/registry.py:189`）。由此，机器、Agent、自动化占位名无论如何都无法越过这条红线——这与第 1 章"机器只做计算与提示，绝不置已立案"的边界声明在代码层完全对齐。

代码层还提供 `DisposalBoard`（`core/disposal.py:49`）作为正兵处置看板，统一封装 `transition / verify / exclude / confirm / file / persist / restore / report` 等方法。它内部持有一个 `ActionExecutor`

(`core/disposal.py:64`)，所有状态变更都经由此执行器，而非直改线索对象，从而在看板这一便利入口也守住了"写路径唯一"原则。每一条状态变更都会生产一条 `StatusAuditEntry` (`core/registry.py:104`)，记录从态、到态、操作人、备注与关联的事件号 (`event_id`)，使处置过程可逐条追溯。

6.2 ActionExecutor: 唯一的写入口

"写路径唯一"是本内核的设计原则之一（见第 1 章全局规范与 `reference-keypoints.md` 第 2 节），其工程落点就是 `ActionExecutor` (`core/action_executor.py:67`)。构造函数签名为 `ActionExecutor(store, pack, access, health, sink)`，五个依赖分别承担存储后端、案件包、权限上下文、运行健康度与可选的 Web 写后端 (`sink`)。它之所以是"唯一"的写入口，是因为 Function 层被强制只读（SQL 白名单只放行 `SELECT / WITH`，见第 5 章），而所有会改变线索状态或产生副作用的动作，无论来自 `run_all` 主链路、`clue_transition` MCP 工具还是两阶段提交，最终都汇聚到这一个类。这种收敛带来的直接收益是：角色校验、状态机校验、参数校验与审计链被集中实现一次，任何新增的写动作都无法绕过它们。

`ActionExecutor` 提供两条执行路径。路径 A 是即时执行 `execute()` (`core/action_executor.py:122`)，用于 `run_all` 与 `clue_transition` 这类"提交即生效"的场景，行为保持既有；路径 B 是两阶段提交（REQ-012），即 `submit → approve → dispatch → mark_confirmed` 的异步闭环，用于需要"机器提议、人类批准"的治理场景。两条路径在落本地变更时共用 `_validate` (`core/action_executor.py:131`) 与 `_apply` (`core/action_executor.py:169`)，校验逻辑因此只有一份真相。

值得把"写路径唯一"的代价与收益说清楚。代价是任何写功能都必须先定义 `actions.json` 中的动作声明，并经由执行器；这比在业务代码里随手 `UPDATE` 一行要啰嗦得多。收益则体现了确定性内核的根本立场：写操作天然带上角色校验、状态机校验与审计链，而不是依赖每个调用方"记得调用审计"。在侦查这类强合规场景下，把约束集中到一处、让旁路无处可走，远比把约束分散到 N 个调用点、再寄希望于代码审查更可靠。这也是为什么 `actions.json` 被归到八段声明中的"写路径"段——写动作本身是一种需要被声明、被编译、被校验的"数据"，而非散落的函数调用。

五步校验是 `_validate` 的主体，任何一步失败都即时抛错，绝不带病进入写操作。第一步是角色校验 (`core/action_executor.py:133`)：当动作声明 `requires_role="human"` 时，若操作人是占位名 (`_is_placeholder_operator`, `core/action_executor.py:28`)，即空串或被 `FORBIDDEN_OPERATORS = {system, ai, assistant, model, bot, auto, llm}` (`core/action_executor.py:25`) 命中的名称，直接抛 `ValueError`。这一步堵死了 Agent 或自动化以"正兵"身份伪造写操作的通道。第二步是必填参数校验 (`core/action_executor.py:139`)：`file` 动作的 `legal_basis` 属于强制字段，缺失即拒，这是"已立案必须援引法定依据"在参数层面的硬约束。第三步是状态机校验 (`core/action_executor.py:147`)，调用 `ClueStatusMachine.validate` 阻断非法迁移。第四步是 `only_from` 显式收紧 (`core/action_executor.py:150`)：在状态机允许的来源之上，动作可进一步声明"仅能从某态发起"，例如固证（已固证）仅允许从查证中发起，待查不可直接越级固证。第五步是权限上下文一致性 (`core/action_executor.py:158`)：非 `system` 会话必须满足 `access.can_transition`，且操作人须与 `AccessContext` 中的主体一致，防止以他人名义代写。

动作本身的声明类型是 `ActionSpec` (`core/ontology.py:170`)，字段含 `name / target_status / allowed_from / parameters / requires_role / side_effects / terminal / only_from / description / title`。这里有一处关键的反向派生设计：`allowed_from`（该动作允许从哪些状态发起）并不在 `actions.json` 里重复声明，而是由 `states.json` 的迁移表反向派生——`_reverse_reach_from` (`core/ontology_loader.py:1661`) 遍历包迁移表，收集所有"一步可达目标态"的源状态，在 `_load_actions` (`core/ontology_loader.py:1710`) 处赋值给 `ActionSpec.allowed_from`。这样做的好处是状态语义只有一个事实来源：改 `states.json` 的迁移表，所有动作的 `allowed_from` 自动随之变化，避免两份声明漂移。副作用同样受白名单约束，`ALLOWED_SIDE_EFFECTS = {set_clue_status, create_decision, merge_entity,`

`dismiss_review` (`core/ontology_loader.py:70`) , 任何未注册的副作用在装载期即报错 (`core/ontology_loader.py:1693`) 。

6.3 两阶段提交

两阶段提交 (REQ-012) 解决的本质问题不是分布式事务, 而是治理诉求: 机器可以提议一个处置动作, 但只有在具名人类批准后才能生效。状态机在 `action_request` 表上推进为 `proposed` → `approved` → `dispatching` → `pending_receipt` → `confirmed` (另含 `dispatch_failed` 与 `dead_letter` 失败分支, 见 `core/action_executor.py:13` 模块文档)。

阶段一是 `submit()` (`core/action_executor.py:192`) : 它先跑完整的 `_validate` 前置校验, 不合格连登记都不做; 通过后把动作登记为 `status='proposed'`, 不执行任何变更, 只写入 `action_request` 草稿。它具备幂等性——幂等键由 `_default_key` (`core/action_executor.py:310`) 生成, 形态为 `f"{action}:{clue}:{sha256(...)[:16]}"`, 相同键重复 `submit` 返回既有 `action_id` (AC5), 避免同一提议被登记两次。阶段二是 `approve(action_id, operator)` (`core/action_executor.py:222`) : 它强制要求 `operator` 为具名正兵, 占位名直接拒 (`core/action_executor.py:224`), 将状态从 `proposed` 推到 `approved`, 并落 `approved_by` 与 `approved_at`, 把"谁批准"钉进记录。阶段三是 `dispatch()` (`core/action_executor.py:240`) : 若动作未被 `approve` 就进入派发, 抛 `NotApprovedError` (`core/action_executor.py:248`), 这保证了"未批准不生效"的硬约束; 正常路径下它先本地提交 (走 `_apply` 改线索状态、触发副作用), 再把回写任务入 `Outbox`, 状态推进到 `pending_receipt`。最后是 `mark_confirmed(action_id, external_id)` (`core/action_executor.py:295`), 在外部业务系统回执唯一业务号后被调用, 置为 `confirmed`。

这套机制最值得强调的工程取舍是"派发 fail-closed" (REQ-G-020)。当 `Outbox` 因故不可用时, `dispatch()` 不会停留在"dispatching 进行中"去假装动作还在路上, 而是显式置 `dispatch_failed` 并落一条 `critical` 级诊断 (`core/action_executor.py:279`)。这是对"未声明即拒绝、失败要留痕"原则的直接贯彻: 任何写操作的中间态都必须可被观测与重放, 绝不能让一个看不到落点的派发被误读为"已发出"。

把"机器提议、人类批准"拆成两阶段, 而不是简单地给 `execute` 加一个 `operator` 参数, 表面看增加了状态与表结构, 实则把治理意图变成了可持久化、可回放的状态机。一个 `proposed` 的草稿即使进程崩溃也不会丢失, 重启后仍能由具名正兵 `approve` 再 `dispatch`; 而 `NotApprovedError` 与具名 `approve` 的强制要求, 堵死了"自动化以人类名义批量生效"的隐患。这种设计在某种程度上是牺牲了即时性来换取可问责性——而侦查系统的第一优先级从来不是吞吐, 是可追溯。对应的写入侧事件也贯穿全链路发布——`action.submitted`、`action.approved`、`action.dispatched`、`writeback.confirmed` 由 `_publish` (`core/action_executor.py:340`) 经事件总线发出, 落盘失败仅留痕不阻断主流程 (REQ-G-004)。

6.4 发件箱、回写与对账

本地提交与外部确认之间隔着一道可靠边界: `Outbox` (`core/outbox.py`, REQ-013)。 `dispatch()` 把待回写 payload 落库, 由 `WritebackDispatcher` 异步取出发送, 状态机为 `queued` → `sent` → `confirmed`, 失败分支 `failed` → `dead_letter`。幂等键稳定为 `wb:{action_id}` (`core/outbox.py:58`), 同一动作重复入队返回既有 `outbox_id`, 确保外部台账不会因为重试而产生重复记录。 `enqueue` 与 `list_by_status` (`core/outbox.py:55`、`:75`) 构成发件箱的基本存取契约。

对外回写的抽象是 `WritebackAdapter Protocol` (`core/writeback.py:48`), 仅含 `dry_run` / `send` / `fetch_status` 三个方法, 且签名中刻意不出现 `conn` / `store` / `board`——从接口层面保证适配器拿不到本体连接, 无法反向读取未授权数据 (AC5)。 `StubLedgerAdapter` (`core/writeback.py:56`) 是本地 JSON 台账的 P0 实现, 支持故障注入 (fail / conflict / 不返回业务号) 以覆盖各类外部异常。一个关键定义是"外部成功"的标准: `Receipt`

(`core/writeback.py:31`) 中的 `external_id` 才是成功凭证——HTTP 200 但无业务号不算成功，`WritebackDispatcher._send_one` (`core/writeback.py:146`) 会 把 它 停 在 `pending_receipt` (`core/writeback.py:169`) ， 绝 不 冒 进 置 `confirmed` (AC4) 。 `DryRunResult` 与 `ExternalStatus` (`core/writeback.py:25` 、 `:40`) 作为不可变结构，承载 `dry-run` 与状态查询的结果。

对账由 `Reconciler` (`core/reconcile.py:36` ， REQ-014) 负责，它面对的是"外部不可靠"这一现实。`MAX_ATTEMPTS = 5` (`core/reconcile.py:32`) ， 退 避 序 列 `BACKOFF_SECONDS = [60, 300, 1800, 7200, 28800]` (`core/reconcile.py:33`) ， 即 1 分钟→5 分钟→30 分钟→2 小时→8 小时的指数退避；达到上限仍失败的转 `dead_letter` 并报告警事件。指数退避而非固定间隔，是为了在外部短时抖动时快速自愈、在外部长时间不可用时又不空耗资源——前紧后松的节奏匹配了"多数故障很快恢复"的真实分布。

特别地，409 冲突不重试，直接进入人工队列 `manual_409` (`core/reconcile.py:56`) ， 因为冲突意味着外部已有记录，重试只会添乱，应交由人类裁决。`pending_receipt` (200 无业务号) 超时后按同幂等键幂等重投，外部不会重复建单。对账差异(本地 `confirmed` 但外部查无此单)只报告、不自动覆盖(`core/reconcile.py:81`) ， 防止外部短暂故障导致本地状态被错误回滚——这条"只报告不覆盖"的克制，正是 `fail-closed` 在跨系统边界上的延伸。它把"本地以何为真相"这个问题回答得很清楚：本地 `action_request` 的状态是权威真相，外部系统的状态只作为参照，绝不以外部缺失来反推本地应当回退。

6.5 file 副作用与决策对象

当处置动作是 `file` (置已立案) 时，除了状态变更，还会产生一个副作用：创建决策对象 (`decision`) 。这由 `_create_decision` (`core/action_executor.py:365`) 实现，它体现了对"Web 快速通道"与"CLI/MCP 既有路径"的兼容分层。若 `sink is not None` (即 `Web state.sqlite` 后端， D-M3-1 设计) ， 决策委托给 `sink.create_decision` ， 不写语义层 `obj_decision` / `lnk_decision_for` ——因为决策是正兵的最终定性产物，不应随语义层 `BUILD` 重建而丢失。否则走 `DuckDB` 路径：先 `ensure_runtime_tables` 保证运行时表存在 (`core/action_executor.py:374`) ， 再 `INSERT INTO obj_decision` 写入 9 列 (`core/action_executor.py:384`) ： `decision_id` (形如 `decision_{int(time.time()*1000)}`) ， `:378`) 、 `target_status` 、 `clue_id` 、 `legal_basis` 、 `operator` 、 `note` 、 `created_at` 、 `metadata` 、 `source_rows` ， 随后 `INSERT INTO lnk_decision_for` (`core/action_executor.py:390`) 建立决策与线索的链接。

把决策对象独立成 `obj_decision` 而非仅存在内存里，意义在于它让"为什么立案、依据什么、谁立的、援引哪条法定依据"成为可检索、可审计、可随案例库复用的第一等数据。它也是 6.8 案例库沉淀的原材料之一。与之配套的审计与事件发布在 `_chain` (`core/action_executor.py:89`) 与 `_publish` (`core/action_executor.py:340`) 中完成：审计链 `AuditChain` 惰性构造并复用同一连接，处置动作 (含受控的已立案终态) 一旦落链，便写入持久哈希链，解决了此前只写内存 `audit_log` 导致的"自检空链假阳性"问题 (REQ-G-025) 。

这里要特别指出主链路上的一个衔接点：在 `run_all.py` 主流程里， `DisposalBoard.persist()` (`run_all.py:294`) 把处置状态落 `DuckDB` 后，紧接着会重新 3 刷新 4 (`run_all.py:296`) 再重算 `report` (`run_all.py:305`) 。这一步看似多余，实则关键—— `obj_clue` 是处置状态的语义层快照，如果 `persist` 后不重建，语义表会停留在"全部待查"的旧状态，导致下游按 `obj_clue` 读取的看板与 `DuckDB` 真实处置状态相互矛盾；而沿用旧 `report` 会让 `by_status` 停留旧快照。该衔接把"写路径的落地"与"语义层的刷新"显式绑定，避免了两层状态漂移这一典型隐患。

6.6 人审闭环与增量重算

人审闭环 (REQ-016) 回答的问题是：当正兵对一条实体对齐候选做出 `accept` 裁决后，语义层与规则结果应该怎么跟着变，且变的范围要可控。核心入口是 `apply_accept` (`core/review_loop.py:119`) ， 流程为：先把归并映射落入受保护

的 `entity_mapping` 表（`core/review_loop.py:22`，`INSERT OR REPLACE` 保证归并结果不被编译器重建清除），再经 `plan_from_review` 计算一跳影响范围，得到 `affected_rules`，随后用 `materialize_changed` 做增量重物化，并只重算受影响规则（`core/review_loop.py:153`、`:164`）。前后两次 `run_rules` 的结果经 `_diff_findings`（`core/review_loop.py:95`）按规则分组做证据集合 diff，产生 `finding.changed` 事件，变化的 `finding` 被标 `needs_review=True`、`review_round=2`，重新进入二次 review（AC5）。这一机制与第 3 章的增量重建内核直接衔接——`materialize_changed` 的行级 diff 与受保护表逻辑即来自那里，本章不再展开其编译细节，只说明“人审裁决是它的一次具体触发点”。

闭环的边界约束同样严格。`reject` / `defer` 不删证据、不触发重建，仅经 `record_feedback`（`core/review_loop.py:70`）写 `review.decided` 反馈事件（AC3）。这一点的工程含义是：人审的“否定”同样是一种可追溯的处置动作，它不改语义层，但被记录下来，使得后续任何审计都能回答“当时为什么没采纳”。幂等由 `review_applied` 表保障（`core/review_loop.py:34`）：同一 `decision_id` 重复应用会直接返回首次结果（`core/review_loop.py:132`），避免事件重放导致重复重算（AC2）。重算只覆盖 `affected_rules`，与本次裁决无关的规则结果保持不变（AC4），这既保证了结果正确，也把计算成本压在最小必要范围——在规则集扩大的未来，这种“只动受影响部分”的增量思维将直接决定系统的可扩展性。

6.7 defer 回捞与异常线索通道

并非所有候选都能立即裁决，暂缓（defer）与异常（anomaly）是两条需要被特殊隔离的线索通道。`DeferredBoard`（`core/deferred.py:166`，REQ-017）管理暂缓任务，登记时必须携带 `wake_conditions`，否则 `ValueError`（`core/deferred.py:80`）——无条件的暂缓等于把线索丢进黑洞。三类唤醒条件按 OR 语义（`core/deferred.py:103` `match_wake`）：`on_dataset`（某数据集/分区到达）、`after`（TTL 到期唤醒）、`evidence_count_gte`（新证据计数达阈值），满足其一即唤醒。一个细腻的护栏是“条件存在但无法解析”的情形：例如 `evidence_count` 字段非整数，`condition_parse_error`（`core/deferred.py:138`）会把它从安静沉睡的 `waiting` 显式转为 `condition_error`（`core/deferred.py:46`）并 `critical` 留痕，而不是让它永远卡在“条件不满足”的假象里（REQ-G-005）。

异常线索通道（REQ-G-019，`core/anomaly_channel.py`）的定位是防止“系统故障被误当成侦查发现”。它把 `run_diagnostic` 中可转化的静默/缺口类诊断——如 `rule_zero_hit` 中仅 `empty_result_suspect` 子类（疑似失效或被规避）、`function_empty_degraded`、`coverage_gap`——转化为与正常 `finding` 同构的异常线索（`clue_from_diagnostic`，`core/anomaly_channel.py:59`）。之所以强调“同构”，是为了让异常线索能复用正常线索的展示与处置看板，而不必另起一套 UI；但它携带的 `source_rows` 恒为空列表，因为它本就没有证据行，溯源走 `diagnostic_ids` 回到 `run_diagnostic`。

这类线索被强制打上不可覆盖的标记：`级别` 恒为“待核实”、`needs_human_review` 恒为 `True`、`is_anomaly` 恒为 `True`（`core/anomaly_channel.py:99`、`:107`）。它绝不参与五间交叉升格（REQ-G-019 AC3）：交叉等级的定义是“非空即命中”，若异常也贡献命中，就会出现“缺得越多等级越高”的荒诞结果。结构上异常线索只是内存/产物中的 dict，绝不写入 `obj_*/lnk_*` 语义表（`core/anomaly_channel.py:18`），任何混合线索流在送交叉/升格前必须经 `non_anomaly` / `partition`（`core/anomaly_channel.py:187`、`:192`）过滤。这条护栏的意义超出技术层面：它把“没查到”与“查到了空白”严格区分——前者可能是数据边界，后者可能是对象刻意规避，机器不应把二者混为一谈并自动给出“无异常”的定性，把判定权留给人类，正是系统“不下定性结论”边界在负向信号上的体现。

6.8 案例库沉淀与四道质量门

当一条线索经过人审闭环被核验后，其可复用的经验应沉淀进案例库（REQ-031，`core/case_library.py`）。`settle_fragment`（`core/case_library.py:92`）是唯一的沉淀入口，它用四道质量门把住入库质量，任一不过即整体拒绝并一次性收集错误（fail-fast 但信息完整）。

第一道是终态门（`core/case_library.py:103`）：线索处置状态必须落在可沉淀的核验结论集合内（默认 `已固证`→`verified`、`已排除`→`excluded`，映射取自 `states.json` 的 `outcome` 声明），待查或查证中一律拒绝——未核验不得入库。第二道是适用条件门（`core/case_library.py:118`）：`pattern` 必须是非空文本、长度不小于 15 字，且必须包含 `rule_id`，确保沉淀的是“适用条件”而非一句空泛结论。第三道是 `legal_basis` 门（`core/case_library.py:127`）：法律依据必须非空，案例沉淀必须可援引。第四道是脱敏门（`core/case_library.py:131`）：`pattern` / `evidence` 经 PII 正则复扫须零命中，且案件知识包 `case_knowledge.json` 中的真实姓名绝不允许出现，必须使用 `当事人#token` 形式——这把脱敏从“生成时”延伸到“入库前再校验一次”，形成双保险。

四门通过后，片段写入 `case_fragment` 表（`core/case_library.py:39`），并携带 `rule_version` 与 `ontology_version` 做版本溯源（AC5）。每次沉淀都会落一条 `AuditChain` 事件（`core/case_library.py:184`），`audit_event_id` 回写进片段行，使案例库同样纳入全局审计链。检索可由 `search`（`core/case_library.py:200`）按 `rule_id` / `outcome` / 关键词组合完成，支撑后续同类侦查的复用与对齐。

四道质量门之所以设得偏严，是因为案例库是“会被未来规则与人员反复引用”的知识资产：没有终态门槛，未核验线索会被当成可靠经验；没有适用条件门槛，沉淀会变成无情境的结论搬运；没有脱敏门槛，真实姓名会顺着案例库二次扩散；没有 `legal_basis`，案例失去可援引性。四门把“可复用”与“可信赖”绑定在一起，而不是先入库再治理——在侦查语境下，后者几乎注定失控。至此，一条线索从产出、处置、闭环到经验沉淀的完整写路径闭合，且每一步都带着角色约束、状态机约束与审计留痕。

第 7 章 安全、权限与可观测性

侦查推演内核处理的是调查数据，对权限、脱敏与可追责有强约束，同时还存在 LLM（大语言模型）与 MCP（Model Context Protocol）两类外部交互面。本章在语义层之上界定权限面，说明“谁能看、谁能做、哪些信息必须遮蔽、每一步如何留痕可回放”，并交代系统故障如何与侦查发现物理隔离。所有强制点均落在具体模块与行号，便于代码审计者逐条核对。本章不重复第 1-6 章已定义的语义层、数据接入、规则引擎与写路径，仅引用其结论。

7.1 权限模型总览

权限面以 `AccessContext`（`core/access.py:79`，frozen dataclass）为唯一载体，贯穿全部五条出口：统一读网关 `OntologyReadGateway`、只读 `FunctionExecutor`、写路径 `ActionExecutor`、MCP Server、以及导出通道。任何一次访问都必须携带一个已通过校验的 `AccessContext`，而不是把“当前用户”散落在各模块局部变量里。frozen 意味着上下文一旦构造便不可改写，配合构造期三重校验，把“身份合法性”前移到对象生命周期起点，也避免了在长调用链中被中途篡改而导致审计失实。

字段包括 `operator`（必填非空）、`role="正兵"`、`case_id="default"`、`purpose=""`、`clearance=1`、`network="local"`。其 `__post_init__`（`core/access.py:92`）执行三重校验：`operator` 为空、`network` 不在 `{local, isolated, web}` 集合、`role` 不在 `ROLE_RANK`，任一不满足即抛 `ValueError`。这三道闸门保证后续所有鉴权逻辑可以信任上下文字段有效，无需再各自判空。系统旁路 `system_context()`（`core/access.py:145`）仅用于内核自身编排（`is_system=True`），依旧过同一构造器，不绕开校验。

与 LLM 调用相关的两道边界在 `context` 上直接暴露：`can_llm_call()`（`core/access.py:113`）在 `network="isolated"` 时返回 `False`；`require_llm_allowed()`（`core/access.py:151`）在其为假时抛 `LLMBlockedError`。这意味着“物理隔离网络禁止任何大模型调用”是代码强制的，而非配置约定。五条出口中凡涉及 LLM 增强的路径，都必须先过此检查；导出通道若要把内容送出隔离环境，同样受 `network` 字段约束。由此，`AccessContext` 不只是一个身份标签，而是把“环境可信度”与“行为能力”绑定的统一凭证。

7.2 角色体系与两把尺子

角色用两把互相独立的尺子刻画，二者在代码注释中明确禁止互比（`core/access.py:40`）。第一把 `ROLE_RANK`（`core/access.py:25`）刻画“行政秩级/动作权限”，取值 `{见习:0, 正兵:1, 偏将:2, 主办:3, human:4, system:99}`；第二把 `ROLE_CLEARANCE`（`core/access.py:44`）刻画“信息密级可见度”，取值 `{见习:0, 正兵:1, 偏将:3, 主办:3, human:3, system:99}`。两把尺子的数值刻意不一致：偏将的 `rank=2` 但 `clearance=3`，`human` 的 `rank=4` 但 `clearance=3`，这本身就是“动作权限”与“信息密级”解耦的具象证据。

之所以不能合并成一把“总级别”，是因为“能不能做某个动作”与“能不能看某个密级”是两个正交维度。动作权限管的是写入、迁移、置状态这类行为能力；信息密级管的是对象属性中敏感字段的可见范围。若合并，会推导出“能看就能改”或“能改就能看”的隐含等式，而侦查场景恰恰要求二者分离：一个高密级可见的分析员未必有写权限，一个被授权发起动作的脚本未必该看到全部明细。注释中“禁止互比”正是对这种耦合的防御——比较权限时只能用 `rank`，比较密级时只能用 `clearance`，混用会在代码评审阶段被拦截。

`jian_clearance_for_role`（`core/access.py:54`）与 `can_see_jian_types`（`core/access.py:59`）进一步把 `clearance` 下沉到“五间”维度：不同间类（因间/内间/反间/死间/生间）带有 `default_clearance`，角色密级低于对象密级即不可见。这把“角色—密级”映射到业务语义层，使权限判定在第 3 章定义的语义对象上自然成立。`system=99` 是唯一

一的旁路值，仅供内核编排使用，业务操作人永远不应以此为 operator，否则会被第 6 章的 `FORBIDDEN_OPERATORS` 拦截。

两把尺子还分别应对不同审计视角：rank 主要回答"这次操作是否被授权"，用于 ActionExecutor 与 MCP 写通道的前置判定；clearance 主要回答"这个角色看到了多少"，用于网关属性级遮蔽与 `can_see_jian_types` 的同类过滤。把二者分开，也使权限变更的影响面可控——上调某人 rank（赋予动作能力）不会自动放大其可见密级，反之亦然。第 1 章禁令③（不置"已立案"）正是 rank 维度的末端约束：human=4 是能置终态的最高业务秩级，而 system 旁路仅用于内核编排、被 `FORBIDDEN_OPERATORS` 排除在状态机操作人之外，二者在代码层互不越权。这也解释了第 18 章已知风险中"两把尺子认知负担"——数值刻意不一致正是正交性的代价，评审时应以注释为准而非直觉对齐。

7.3 三级鉴权

鉴权在语义层之上分三级：`PolicyEngine`（`core/policy.py:49`）提供 `check_object`（对象级，`policy.py:86`）、`check_link`（链接级，`policy.py:102`）、`can_read_property / mask_value`（属性级，`policy.py:122 / 129`）。对象级与链接级查不到声明即抛 `PolicyDeniedError`（`policy.py:26`），属性级则进入脱敏而非拒绝——这是"拒绝读取敏感对象"与"返回但遮蔽敏感列"的区分。三级对应"能否访问这张表 / 能否沿这条关系 / 能否看到这一列"的逐层收紧。

这种分层不是冗余，而是对应语义层的三类访问原语：对象是最粗的边界（一张业务表能否被这个角色读），链接是关系边界（沿某条边能否遍历），属性是最细的边界（某一列值能否以明文/半遮出现）。网关作为统一读入口（REQ-002），把三级鉴权收口在 `objects/links/view/prop_indicator` 等少数方法内，调用方无法通过绕过网关直查 `obj_*/lnk_*` 来逃避——第 1 章禁令①（直查拦截 `_FORBIDDEN_TABLES`）在存储层兜底，与权限面的网关收口形成纵深防御。三级鉴权因此既是策略表达，也是不可替代的代码执行路径，而非可被上层随意绕过的约定。

对象级策略在 `policies.json` 中以 `object_policies` 字典承载（`policy.py:79`）。`check_object` 先取 `object_policies.get(name)`，取不到即 `raise PolicyDeniedError`（`policy.py:90`、`:97`）；链接级同理（`policy.py:108`、`:113`）。语义视图 `view()`（`core/gateway.py:135`）额外校验 `access.role ∈ view.roles`，不在名单内抛 `ViewAccessDenied`（`gateway.py:152`），但仍按 `base_object` 走对象级+属性级策略——视图只是投影，不绕过底层权限。这样即便声明了一个宽松视图，敏感列的遮蔽仍由 `base_object` 的属性策略兜底。

7.3.1 三处 fail-closed 强制点

fail-closed（未声明即拒绝）在本节有三处强制点，构成"安全默认态"的骨架。第一处：`policies.json` 缺失时，`PolicyEngine.__init__` 令 `self.object_policies = {}` 且 `self._missing_file = True`（`policy.py:61`、`:63`），于是任何对象查询都查不到策略而被拒绝，而非放行。第二处：`schema_version != 2` 时装载期直接 `raise ValueError`（`policy.py:69`，对应 REQ-G-016），防止旧格式声明被误当作有效权限。第三处：声明中出现的角色集若不在 `ROLE_RANK` 内，同样 `raise ValueError`。

这三处强制点的工程意义在于：当新增一个对象却忘记配策略、或策略文件损坏、或声明格式漂移时，系统会拒绝访问而不是悄悄放行。对被审计系统而言，"新增即默认拒绝"是可追责的前提——它把配置疏漏暴露成运行时失败，而非埋成数据越权隐患。这也与第 4 章的数据接入路径呼应：新数据源接入的第六步明确要求在 `policies.json` 补策略，漏配会在运行时被 fail-closed 捕获，而不是等到越权访问才暴露。

7.4 脱敏策略

属性级脱敏由 `_MASKS` 映射（`policy.py:46`）驱动，含 `partial` 与 `full` 两档。`mask_partial`（`policy.py:38`）保留前 3 位与后 4 位、中段以 `*` 填充，例如 `310****1234`；长度不足 8 时整体遮蔽（`policy.py:39` 注释），因为过短的值即便只露前 3 后 4 也可能被反推。`full` 档直接返回 `***`

(`policy.py:46`)。 `mask_value` (`policy.py:129`) 按属性声明的 `mask` 字段选择函数，缺省走 `full`，即“未显式声明可部分展示的列一律全遮”。

脱敏不仅作用于逐行明细，也覆盖聚合统计。网关的 `value_profile` (`core/gateway.py:221`) 与 `distinct_values` (`core/gateway.py:279`) 在返回 `samples/min/max` 等统计时，会连同原始值样本一并遮蔽——避免“不直接返回明细、却把样本泄露出去”的旁路。这与第 1 章禁令②（不把原始明细搬进上下文）形成闭环：Function 只聚合与溯源 ID，聚合里的样本同样受 `mask_value` 约束。`prop_indicator` (`gateway.py:300`) 等派生指标亦走同一脱敏管线。

需要强调的是，脱敏是属性级策略的结果而非调用方自觉。`apply_row_masks` (`policy.py:136`) 对整批行统一施加，调用方无法“按需跳过”某一行：`can_read_property` 返回布尔，决定该行是否进入脱敏前的可见判定。这样即便上层代码忘记遮蔽，网关返回的列集已经过策略裁剪，原始敏感值不会以明文形态出现在出口。对于 `value_profile` / `distinct_values` 这类会回显样本值的接口，遮蔽在返回前统一完成，杜绝了逐接口遗漏。

7.5 审计哈希链

审计由 `AuditChain` (`core/audit.py:115`) 实现，是一条 SHA-256 链式结构。链首 `_GENESIS_HASH = "0"*64` (`audit.py:21`)，之后每条事件在 `append` (`audit.py:178`) 时取上一条的 `signature` 作为 `prev_hash` (`audit.py:188`)，再对“全部字段 + `prev_hash`”计算 `sha256` 得到本条 `signature` (`audit.py:96`、`:100`)。因此任何一条历史记录被篡改，都会使其后续整条链的 `prev_hash` 衔接失败、签名重算不一致，篡改无法局部隐藏。

链完整性校验由 `chain_verify` (`audit.py:223`) 与 `chain_integrity` (`audit.py:256`) 完成，后者会列出 `broken_links`（断链或签名不一致的事件 ID），供审计者定位疑似篡改或丢链。`signature` 算法 `_compute_signature` 是双后端共用的纯函数，因此 `backend="duckdb"`（案件版本库）与 `backend="sqlite"`（Web 业务状态库 `state.sqlite`）下，同输入产生等价的 `signature` 与 `root_hash` (`audit.py:378`)，两条链路可交叉验证——Web 侧写入的状态库与 CLI/MCP 侧写入的案件库能相互印证，防止任一侧单独被绕过。

版本锚点缺失是现实中的常见情况——`sqlite` 后端没有 `meta_ontology_state` 表，拿不到本体版本号。此时 `append` 不崩、不改动表结构，而是在 `after` 状态内打 `anchor_status=missing` (`audit.py:194`)、并落 `version_anchor_missing` 诊断 (`audit.py:168`、`:173`)。这保证了“锚点缺失”是可观测的降级而非静默通过，符合第 2 章原则“失败要留痕，不许静默”。由于不改表结构，存量审计记录在版本机制演进后仍可继续追加，不会因锚点格式变化而作废。

链的抗篡改性还依赖 `seq` 自增序与签名冗余：即便攻击者尝试替换中间某条记录，由于 `prev_hash` 引用上一条签名，后续每条记录的签名都会失配，`chain_integrity` 一次性可定位首条断点。双后端等价则意味着 Web 侧 (`sqlite`) 与内核侧 (`duckdb`) 任一侧单独被改都无法自治——交叉校验任一端即可发现另一端被篡改，这正是把审计做成“双向可证”而非“单点可写”的设计意图。对于离线部署场景，即使没有 Web 状态库，内核侧 `duckdb` 审计链本身也已具备逐条可验的完整性，不依赖外部组件即可对外举证。审计链与事件总线（7.6）在职责上互补但都不进入 `findings` 主列表，这与 7.7 的物理分离原则一致：审计记录的是“谁改了什么状态”，事件是“发生了什么动作”，二者都属于系统自身行为的证据，而非侦查发现。把系统行为与侦查发现分库分表，既避免健康度噪声污染交叉升格，也让审计链在司法鉴定场景下可被独立提取与验证。

7.6 事件总线

事件总线 `EventBus` (`core/event_bus.py:92`) 基于 DuckDB 三张表：`event_log`（自增 `seq` 主序）、`event_dead_letter`（死信）、`event_idempotency`（幂等键）。系统中目前登记 17 类事件，覆盖构建、规则运行、动作派发、审计、诊断等生命周期节点，使各模块能以解耦方式广播“发生了什么”而不必了解消费者。`publish`

(`event_bus.py:114`) 将 `payload` 的 `sha256` 作为 `payload_hash` (`event_bus.py:125`) 写入, 既可用于完整性核对, 也作为幂等去重的输入。

消费失败不会丢失: `handler` 异常时被写入 `event_dead_letter` (`event_bus.py:162`、`:231`), 后续可盘点重投, 调用方原先 `except: pass` 的静默吞错被改为落 `event_publish_failed / event_dead_letter_summary` 诊断 (`run_health.py` 对应 `kind`)。同一 (`idempotency_key, handler_name`) 用 `INSERT OR IGNORE` 去重 (`event_bus.py:147`), 保证幂等消费。环检测 `detect_cycle` (`event_bus.py:240`) 沿 `causation_id` 逐跳回溯, 当深度超过 `max_depth=10` 判定为循环 (`event_bus.py:253`), 避免因因果链无限递归导致重放雪崩。

重放 `replay` (`event_bus.py:179`) 从指定事件之后顺序重投所有事件, 并为每个事件生成 `replay-` 前缀的新幂等键 (`event_bus.py:207`), 确保重放与首次执行互不冲突、可重复演练。`list_events` (`event_bus.py:258`) 支持按类型与条数检索, 便于事后取证。事件总线与审计链是互补的两套可观测设施: 审计链记录"状态如何变化且不可篡改", 事件总线记录"发生了哪些动作、可否重放"。二者都不依赖 LLM, 在内核零依赖前提下即可运行, 满足离线部署要求。

17 类事件的设计遵循"凡是状态跃迁与可观测动作都留事件"的原则, 使下游消费者(如审计、降级、人审闭环)可用订阅而非轮询的方式响应。例如 `ontology.materialized` 事件驱动人审闭环(REQ-016)的增量重算, `dispatch_failed` 类事件进入 `event_dead_letter` 后可被对账模块(REQ-014)重试直至 `manual_409`。`detect_cycle` 的 10 跳上限不是业务规则而是防呆: 正常因果链深度远低于此, 超过即说明 `causation_id` 回填出现环路 bug, 应告警而非无限重放。事件总线的存在, 让"可观测性"从被动查日志升级为主动的、可重放的事件流, 也为调试与取证提供了时间有序的因果链。

7.7 运行健康度与降级协议

运行期所有非致命异常与降级一律落 `run_diagnostic` 表, 由 `RunHealth` (`core/run_health.py:79`) 统一管理, `record` 方法校验 `kind` 必须在 `KINDS` 元组内 (`run_health.py:105`), 否则抛 `ValueError`, 防止拼写错误的诊断种类悄悄入库。`KINDS` (`run_health.py:28`) 实际登记 28 类诊断, 可归为几族: 构建期降级(脏值/缺列/隔离/复合值/清洗剔除率)、实体解析降级、事件/派发生命周期失败、覆盖与审计缺口、质量门与合规告警、以及手动运行印记。所有接入点签名均兼容 `health=None`, 此时经 `get_health` (`run_health.py:261`) 返回 `NullRunHealth` (`run_health.py:236`) 做空操作, 保证既有不传参调用零行为变化。

与 `findings`(侦查发现主列表)物理分离是本节的核心设计决策。诊断进 `run_diagnostic`, 枚举为产物的"健康度"小节; 语义层重建编译器只 `DROP obj_*/lnk_*` (`run_health.py:16` 注释), 不会触碰运行期表。之所以必须分离, 是因为系统故障(如脏值降级、缺列、派发失败)若在 `findings` 里, 会被当成一条真实的侦查发现, 进而触发第 5 章的五间交叉升格——一次数据接入异常可能错误地"升格"为立案依据候选, 后果严重且难以回溯。物理隔离使"系统说不清楚"与"系统发现了什么"成为两条不可混淆的数据流。

降级协议的整体基调是"降级但不静默、留痕但不污染": 每一项降级都带 `severity` (`info/warning/critical`, `run_health.py:59`) 与可下钻样本, 且样本本身经脱敏(如 `compliance_violation` 只记对象.属性+代理键+违规码, 原始值不入库)。这要求所有接入点默认传入 `health`, 而历史遗留调用以 `NullRunHealth` 兜底, 不会因缺失 `health` 而抛错。`diagnostic_run` (`run_health.py:56`) 保证即便是零问题的运行也会留一条 `info` 印记, 使"无诊断"本身也可被观测、可被证明不是"静默吞错"。

7.8 内容寻址溯源

溯源由 `core/row_uri.py` 提供内容寻址能力, 使"某条 finding 来自哪一行原始数据"可被精确回捞。行 URI 形态为 `dataset@version#partition/rowid` (`row_uri.py:6`、`:81`): `dataset` 标识源表, `version` 为本体版本(hex), `partition` 为数据分区(`bootstrap / 增量 partition_id / incremental`), `rowid` 为行内容地址

`sha256(dataset + 有序 col=value)[:16]` (`row_uri.py:12`)。内容不变则 `rowid` 不变, 旧 URI 永远 resolve 回旧内容, 这正是"内容寻址"相比行号寻址的抗漂移之处。

构造与解析由 `make_row_uri` (`row_uri.py:71`) 与 `parse_row_uri` (`row_uri.py:84`) 完成, 二者均强制各段非空、不含保留分隔符、`version/rowid` 须为 hex, 形态非法抛 `MalformedUriError`。`row_id_for` (`row_uri.py:109`) 与 `uri_for_row` (`row_uri.py:122`) 从一行数据算内容地址; `snapshot_source_rows` (`row_uri.py:145`) 在构建期把行内容写入 `row_archive`、把构建索引写入 `row_build_index` (两表定义见 `row_uri.py:16`、`:20`), 均用 `INSERT OR IGNORE` 去重, 因此同内容多次构建不会重复归档。

回捞由 `resolve_row_uri` (`row_uri.py:252`) 完成: 先据 `row_build_index` 校验该行是否属于指定 `version` (`row_uri.py:281`, 否则报"不属于该版本"), 再从 `row_archive` 取 `content` (`row_uri.py:285`)。这实现了第 1 章禁令②(不把原始明细搬进上下文)——Function 只返回聚合结果与溯源 ID, 需要细看时再用 URI 回捞, 且回捞出的仍是受密级约束的数据。内容寻址的另一个好处是跨构建版本的可对比: 同一 `rowid` 在多个 `version` 的 `row_build_index` 中出现, 即可证明"这行原始数据在 N 次构建里都被消费过", 为溯源与对账提供稳定锚点。

7.9 规则度量与推翻率告警

规则运行时度量由 `core/metrics.py` (REQ-030) 承载。`ensure_rule_run_metric` (`metrics.py:34`) 建表; `record_run` (`metrics.py:77`) 记录每次运行逐规则结果; `verdict_backfill` (`metrics.py:101`) 回填人工裁决。关键设计是 `rule_version` (`metrics.py:66`) = `hash(rule_id + rule_text + 排序后的 params)`, 使指标按"规则语义版本"而非"规则 ID"分层——同 ID 但判据/参数变更会被视为不同版本, 避免历史指标错配到新规则。版本分层后, 调整阈值或改措辞都不会污染旧版本的表现基线。

聚合视图提供三个比率: `hit_rate` (`metrics.py:133`, 命中/评估)、`precision_estimate` (`metrics.py:141`, 以人工裁决为正的精确率估计)、`override_rate` (`metrics.py:150`, = `override_count / evaluated`)。其中推翻率 (`override_rate`) 直接对应质量信号: 当人工裁决反复推翻某规则命中, 说明该规则在当前案件上产生过多误报或已不适用。`alert_override_rate` (`metrics.py:252`) 遍历指标, 对超阈规则落 `override_rate_alert` (warning) 诊断; 阈值取自 `thresholds.json` 的 `alerts.override_rate_max`, 读不到回落 0.5 (`metrics.py:246`)。把"规则被推翻"做成可告警信号, 使规则质量从一次性评审变成持续可观测的指标。

度量本身也受权限约束: `list_metrics` (`metrics.py:165`) 按 `access` 过滤, 敏感规则 (如 R3/R4) 对低权限调用方只返回数值量级摘要 (`metrics.py:230` 注释), 不暴露可被反推的明细。这样"规则表现好不好"可观测, 但"哪条线索被哪条规则命中"的敏感关联不越权外泄。`precision_estimate` 依赖人工裁决回填, 在裁决样本不足时返回 `None` 而非臆造数值, 避免用稀薄样本给出误导性高精度。

推翻率告警还反向驱动规则生命周期: 当某规则 `override_rate` 持续超阈, 结合 `precision_estimate` 走低, 可作为将其移入 `retired` 状态 (见参数治理 REQ-032 的 draft/shadow/production/retired 四态) 的量化依据, 而非仅凭主观判断。度量因此不只是"复盘报表", 而是把第 5 章规则引擎的"判据质量"纳入持续反馈闭环——规则上线后表现可观测、可比较、可退役, 避免僵化的误报长期污染 findings。版本分层 (`rule_version`) 则保证这种比较始终在同语义版本内进行, 不会把"改判据后的新规则"与"旧规则"的命中率混为一谈。

7.10 MCP 边界与 LLM 治理

MCP Server 暴露 13 个工具 (`scripts/mcp_server.py:_TOOL_IMPL`, `mcp_server.py:933`), 按风险分三档。第一档为纯只读侦察类, 无副作用、只附摘要: 状态源强制走 DuckDB 而非 JSON 快照的 `clue_list` (`mcp_server.py` 注释, 避免读到过期镜像)、`scan_anomaly` / `cross_jian` / `graph_overpass` / `function_list` / `function_invoke` / `rule_list` / `review.list_pending`

(不附 `evidence` 明细) / `action.status`。第二档为受控写, 必须经人工坐落: `clue_transition` 拒 `system/ai/agent` 操作人、置"已立案"必带 `legal_basis` 且只能从"已固证"迁移 (`mcp_server.py:541`、`:583`), `run_pipeline` 需 `confirm=true` (`mcp_server.py:625`)。第三档为 Agent 唯一写通道 `review.submit_proposal` (`mcp_server.py:302`、`:844`), 只建 `status=draft` 提案、永不自动生效、绝不改线索状态。

人机边界由代码而非提示词保证: `clue_transition` 中若 `operator` 属 Agent 身份直接报错 (`mcp_server.py:555`); `review.get_evidence` 对正兵及以下只返回类型与计数摘要, 阈值取 `ROLE_RANK["偏将"]` (`mcp_server.py:735`)。所有返回体经统一封装强制挂 `needs_human_review` / `定性_policy` / `disclaimer` (`mcp_server.py:377` - `:379`), 使"AI 不给出定性结论"成为协议层事实而非模型自觉。即便上游模型越权生成了定性表述, 网关封装层也会附上 `定性_policy` 声明, 下游消费方据此识别这是候选而非结论。

LLM 治理分五块。其一 `core/llm/redact.py` (REQ-038) 出网脱敏, PII 正则顺序敏感: 身份证 `[?<!\d)\d{17}[\dXx](?!\\d)` 必须先于银行卡 `\\d{16,19}` (`redact.py:33` - `:40`), 否则 18 位身份证会被 16 - 19 位银行卡规则吞掉, 导致身份证末位被当作卡号中段保留而前段泄露; 此外未走脱敏通道的输入 (缺 `redaction_hash`) 或复扫仍检出 PII 一律禁止出网 (`redact.py:360`、`:369`)。其二 `core/llm/guard.py` (REQ-039) 提示注入防护: `scan_text` / `wrap_untrusted` (把外部内容包成 `untrusted_content` 帧) / `scan_bundle` / `sanitize_candidate` / `assert_no_status_change` (候选含状态变更即拦截, `guard.py:198`) / `raw_evidence_fragment` (原始证据片段只供人工、不进模型上下文, `guard.py:224`)。

其三 `core/llm/explain.py` 的 `_QUALITATIVE_WORDS` 定性词黑名单 (`explain.py:29`), 含"涉嫌/确认/违法/立案"等, 命中即剔除或拒绝输出 (`explain.py:153`、`:184`), 把"不下定性结论"做成编译期正则而非依赖模型守纪律。其四 `core/llm/fallback.py` (REQ-040) 一键降级 + 影子模式: `llm_enabled=false` 即零模型调用 (`fallback.py:29`), 影子模式仅产 `proposal`/日志/审计、不进生产 `finding`; `LLM_OWNED_TABLES` (`fallback.py:18`, `{proposal, llm_call_log, audit_chain}`) 界定 LLM 可写的表集合, `shadow_diff` 断言其余表行数零变化 (`fallback.py:69`、`:78`), 从存储层证明"LLM 只写它自己的表"。其五 `core/proposal.py` (REQ-033) 对提案做七项硬校验 (信封 `jsonschema` / 函数白名单 / 参数强类型 / 证据 URI 合法等, `validate_proposal` 于 `proposal.py:148`), `confidence_only_sorts` (`proposal.py:241`) 断言"不同 `confidence` 下命中集合不变"——`confidence` 只用于排序, 绝不改变定性。

综上, MCP 与 LLM 两层都遵循同一原则: 模型是可选增强而非决策主体, 所有写动作、所有定性、所有出网文本都受代码级闸门约束, 且任意一层失效都能一键降级到确定性内核。第 1 章三条禁令与第 2 章六条原则在本章得到可审计的落点: 权限 fail-closed、脱敏强制、审计不可篡改、故障与发现物理分离, 共同构成内核的安全底座。

三档分级的底层逻辑是"写能力逐级收窄": 只读档零副作用、受 DuckDB 真值源约束; 受控写档把动作锁在人工决断 (`legal_basis/confirm`) 之内; Agent 写档则被压缩到只能投递草稿。影子模式进一步把 LLM 的"产出"与"生效"切开——`shadow_diff` 以表行数快照断言 LLM 仅写入 `LLM_OWNED_TABLES`、其余生产表行数零变化, 从存储层证明降级可逆、增强可回退。这种"能力越小、闸门越硬"的收敛结构, 使外部交互面无论多宽, 最终都收敛到确定性内核这一唯一可信基; 即便 MCP 工具或 LLM 层被完全关闭, 侦查推演仍能在零外部依赖下完整运行, 满足物理隔离部署的硬约束。

第 8 章 质量保障、部署与演进路线

本章是《孙武侦查官》确定性侦查推演内核技术设计文档的收尾章。前七章已分别界定了项目定位与三条禁令、总体架构与关键架构决策（ADR）、语义层编译内核、数据接入与治理、规则引擎与 Function 计算层、线索生命周期与写路径、安全权限与可观测性。本章不再复述上述技术细节，而是把视角收束到工程闭环上：系统如何被验证、如何被部署、当前的真实能力边界在哪里、以及下一步合理的技术演进方向。需要强调的是，本章对已知缺陷与技术债一律如实盘点，不回避、不粉饰——这既是给代码审计者的设计依据，也是对后续维护者的诚实交代。

8.1 测试体系

本项目的测试体系以 `run_tests.py` 的 `GROUPS` 注册表为唯一权威入口，当前实际注册 121 个测试组，对应 `tests/` 目录下 98 个测试文件。这一设计的最大价值，在于建立了“需求编号 → 测试组 → 代码实现”三者可互查的闭环：每个测试组在注册时即绑定其对应的需求编号，使得任意一条需求都能反查到落地它的测试，也能从一次测试失败反推受影响的需求范围。这种可追溯性是本内核“确定性优先、可回放、可追责”核心命题在质量保障层面的具体落地，也是本项目最值得肯定的工程实践之一。

需求编号与测试组的映射覆盖了完整的规格谱系：`REQ-001` 至 `REQ-046` 为内核主体需求；`REQ-G-001` 至 `REQ-G-024` 为降级与治理（Governance）类需求；`REQ-D-001` 至 `REQ-D-022` 为数据接入与治理（Data）类需求；此外还有 `REQ-P M1~M6` 与 Web 端 `M1~M6` 两组独立编号空间。任何一组测试都可在 `run_tests.py` 的 `GROUPS` 字典中按组名检索到其标注的需求编号与执行命令，审计者无需阅读分散的测试文件即可建立整体映射。

在 121 组测试中，有两条静态红线被强制纳入 CI，作为不可绕过的硬性门槛。其一是 `audit` 组（`run_tests.py:83`），执行 `scripts/audit_straight_sql.py --fail-on-violation`，对源码做“直查业务表”静态扫描，专门守卫第一章所述禁令①（不自己写业务 SQL）。其二是 `guard` 组（`run_tests.py:79`），执行 `tests.test_store_guard`，验证 `core/store.py` 的直查拦截逻辑在运行时确实抛出 `DirectSourceAccessError`。两条红线一静一动、互为补充：静态扫描防止新增代码在编写期引入直查，运行时拦截防止既有绕过通道被误用，二者共同构成了禁令①的双保险。

针对 MCP 端到端通道，项目提供 `scripts/mcp_client_test.py` 作为冒烟测试，内含 69 项断言，覆盖 13 个 MCP 工具，并且明确包含三条红线用例（即系统/AI/agent 操作人拦截、置“已立案”必带 `legal_basis`、只读 Function 不可写）。日常执行支持两个开关：`--fast` 跳过端到端（e2e）测试以加速本地循环，`--only <组名>` 则支持单组复跑，便于在修复单个需求时精确定位。需要特别提醒的是，`AGENTS.md` 作为 agent 的第一入口，其第 59 行仍写有“70 组测试，必须全绿”的硬编码数字——这已是明确的文档漂移，详见 8.5 第 1 项与 8.7 的整改要求。

8.2 测试策略分层

本内核的测试并非平面堆砌，而是按风险面分层组织，使每一层各自承担不同的验证目标。单元测试聚焦单一模块的内部契约，例如 `core/ontology.py` 的代理键分配、`core/access.py` 的角色判定，它们运行快、隔离强，是回归防护的第一道关。契约测试关注模块间的接口约定，尤其是语义层声明（JSON）与编译产物（DuckDB 表结构）之间的一致性——链接物化时的“边属性与输出列对账不一致即硬失败”就是一类契约断言。集成测试则把存储层、语义层、规则引擎串起来，验证跨模块的端到端数据流，例如从原始 Parquet 经 `build_ontology` 到 `run_rules` 产出的 `finding` 是否携带正确的 `rule_id` 与降级标记。

端到端测试主要落在两条路径上：其一是 8.1 已述的 `scripts/mcp_client_test.py` 这一 MCP 冒烟通道；其二是 `run_all.py` 主流程的整链验证，覆盖从数据生成、接入、语义层构建到线索输出的完整产物。静态扫描层则独立于运行

期测试，由 `audit` 组（`scripts/audit_straight_sql.py`）与 `guard` 组（`tests.test_store_guard`）两条红线承担，在 CI 中先于或并行于功能测试执行，确保禁令在代码进入仓库前即被守住。

值得单独说明的，是贯穿全仓的降级协议（REQ-G 系列）测试，约 20 个测试组遵循统一协议：“降级但不静默，留痕但不污染”。在 `run_tests.py` 中这些组被显式分波注释（如“第一波：底座留痕”“第二波：治理口径”“第三波：声明化”），各自验证某一降级路径是否既保留了诊断痕迹、又不污染 findings 主表。这类测试的存在，使得 8.5 将讨论的“图算法能力薄”“Windows 图库不可用”等能力缺口，在工程上已被降级协议妥善兜底——能力缺失时系统降级而非崩溃，这正是确定性内核的韧性来源。

8.3 环境与部署约束

本内核的部署形态高度依赖运行环境，必须在项目早期就厘清 WSL2 与 Windows 原生之间的差异，否则极易在交付环节踩坑。开发主环境为 WSL2（Ubuntu-24.04），Python 虚拟环境位于 `/root/.venvs/inves`；由于仓库依赖较多且网络受限，pip 安装必须使用国内镜像（默认清华源，遇到 403 时切换阿里云）。这一约束看似琐碎，但它是保证可复现构建的前置条件，应在环境文档与 CI 基线中固化。

一个隐蔽但高频的网络问题是 WSL2 的 mirrored 模式断网现象：表现为 DNS 解析正常、但所有外网 TCP 连接全断。其根因是 Windows 端 TUN 网卡下发的两段兜底路由 `0.0.0.0/1` 与 `128.0.0.0/1` 被镜像进了 WSL 路由表，覆盖了本应直连的本地网段；只需删除这两条路由即可恢复，本机已常驻 `fix-wsl-routes` 服务自动处置。该问题不影响离线内核本身，但会阻断镜像源与包安装，属于部署层面的“环境债”。

图库（L4，LadybugDB）的可用性呈明显的环境矩阵特征。在 WSL/Linux 环境下图库可用，但需手工将 `libduckdb.so` 放置到 `~/libdb/extension/<版本>/<平台>/common/` 目录——直接放入 `/usr/local/lib` 是无效路径，这是 LadybugDB 扩展加载机制的硬性约定。在 Windows 原生环境下，图库实质不可用；LadybugDB 官方 CI 不构建 Windows 版扩展，项目被迫走 CSV 中转方案；同时 `ATTACH DuckDB` 在 Windows 下也不可用。这意味着同一套代码在两类环境下的图能力 completeness 并不对等，相关差异必须在部署说明中显式标注，避免使用者误以为 Windows 下具备完整图谱能力。

离线部署能力是本内核的核心卖点之一，也是其可进入物理隔离网络的底气。内核纯离线、零 LLM 依赖、零 API Key；当 `AccessContext` 的 `network="isolated"` 时，`core/access.py` 的 `can_llm_call()` 返回 False，LLM 调用被强制关闭（对应 REQ-040 的一键降级开关与影子模式）。这使得在同一份代码与本体声明下，既可运行带 LLM 增强的联网形态，也可无缝切换为完全离线、确定性的隔离形态，二者差异仅由运行期上下文决定，不引入第二套代码路径。

8.4 性能与容量

本内核虽以确定性为首要目标，但其面向的侦查数据往往体量可观，因此增量重建与采样预演两道机制直接决定了大规模场景下的可用性。增量重建由 `core/rebuild_planner.py` 负责计算影响范围，其批处理阈值常量 `DEFAULT_BATCH_THRESHOLD=5000` 决定了“一次重建最多纳入多少对象/行”的边界——超过该阈值时规划器会切分批次，避免单次事务过大导致的锁竞争与内存压力。该值当前为编译期常量，在 8.7 的演进讨论中将建议其改为可配置项，以便针对不同硬件规格调优。

在数据接入后的主流程中，系统会先执行 1% 采样预演，由 `core/sampling.py` 的 `SamplingPreflight(store, sample_ratio=0.01)` 承担。该预演并非装饰性步骤，而是带有明确判定分支的方向性检查：若采样命中率 $\geq 5\%$ ，则判定数据充分、建议走全量；若落在 1%~5% 区间，则建议扩大采样比例后再决策；若 $< 1\%$ ，则直接判定当前数据方向不成立、不建议继续全量计算。这一机制把“数据是否足以支撑后续重计算”这一原本依赖人工经验的问题，变成了机器可复现的前置判据，符合内核“把经验手艺变成确定性过程”的总命题。

性能回归则由独立的基准测试组 `benchmarks` 守护，对应 `tests/test_benchmarks.py`（REQ-045）。该组提供可重复的性能基线，使得后续任何涉及语义层编译、规则引擎或图构建的性能改动，都能被量化对比而非凭感觉判断。在演进路线中，语义层增量性能优化（8.6 中期）将直接依赖该基准组的长期数据积累。

8.5 已知缺陷与技术债

本节如实盘点当前已知的七项缺陷与技术债。每项均按“现象 → 影响 → 建议”三要素展开，力求坦诚而不消极——每一项都附带可执行、可排期的改进建议，而非仅停留在问题陈述。

1. 文档漂移（Documentation Drift）。现象：`AGENTS.md` 第 59 行写有“70 组测试，必须全绿”，但实际 `run_tests.py` 的 `GROUPS` 注册表已增长至 121 组；`AGENTS.md` 是 agent 修改代码的第一入口，硬编码的过时数字会直接误导使用者。影响：维护者可能以“跑完 70 组即全绿”为完成标准，从而漏掉半数以上的真实测试覆盖，侵蚀 8.1 所述“需求—测试—代码”互查闭环的可信度。建议：立即将 `AGENTS.md` 中的硬编码数字改为“以 `run_tests.py` 的 `GROUPS` 注册表为准，禁止写死组数”，并在 CI 中加入一条断言，校验 `AGENTS.md` 引用的组数与 `GROUPS` 实际长度一致，从机制上杜绝再次漂移。

2. 图算法能力薄。现象：当前图库仅实现两跳定长路径、变长跳邻居、degree（度）统计三类基础能力；Louvain/LPA 社区发现、PageRank、中介/接近中心性、最短路径、k-core 均未实现。影响：团伙识别只能依赖人工观察两跳邻居，无法在算法层面自动发现潜在社群与关键节点，限制了本内核在复杂关系网络中的研判深度。建议：按价值密度排序优先补齐 Louvain 社区发现与最短路径两类（见 8.6 中期），其余中心性指标作为后续增强，并在补齐前于文档中明确标注“图算法为辅助提示、非正式结论”。

3. Windows 图库不可用。现象：LadybugDB 官方 CI 不构建 Windows 版扩展，Windows 原生环境下图能力实质残缺，只能走 CSV 中转；`ATTACH DuckDB` 同样不可用（见 8.3）。影响：跨平台交付时，同一本体声明在 Windows 上的图谱完整性低于 WSL/Linux，若使用者在 Windows 下误用图能力会得到降级或缺失结果。建议：在部署文档中以“图库可用性矩阵”明确标注各环境能力差异，并在 `GraphBackend.available=False` 时统一返回带 `degraded` 标记、且不含图能力的降级产物，确保不静默。

4. 规则覆盖偏窄。现象：现有检测规则仅 R1–R6，其中资金维度占 3 条（R1/R2/R6），通讯/行为/关系各 1 条（R3/R4/R5），时间维度仅 R6 且依附于资金判据。影响：多维（资金/通讯/行为/关系/时间）能力严重不均，时间维度的独立研判基本空白，难以支撑多场景的均衡覆盖。建议：短期优先补充时间维度独立规则与行为维度扩展规则，使五维至少各有稳健的基线覆盖；同时建立“维度覆盖率”检查（见 8.7）。

5. 角色双尺认知负担。现象：`core/access.py` 中 `ROLE_RANK` 与 `ROLE_CLEARANCE` 数值不一致（如偏将 rank=2/clearance=3，human rank=4/clearance=3），虽注释明确要求“禁止互比”，但两把尺子并存本就是典型易错点。影响：新接手者在实现权限判断时，极易混淆应使用 rank 还是 clearance，引入越权或误拒缺陷。建议：在不破坏既有语义的前提下，为两把尺子补充更严格的类型约束或单测守护；长期考虑在代码层以命名枚举（enum）替代裸整数，从语言层面杜绝误用。

6. 启发式扫描误用风险。现象：`sensitive_scan`（默认 `hit_ratio=0.3`）与 `unit_scan`（中位数差 $\geq$ `ratio_threshold=10000`）是启发式质量扫描，设计上“只告警不阻断”是正确的（见 8.2 降级协议）。影响：风险在于使用者可能把告警当作权威结论，对疑似敏感列或单位口径异常直接定性。建议：在输出物与 UI 中明确标注此类结果为“启发式提示、需人工确认”，并在文档与告警文案中固化“告警非判定”的措辞，防止下游误用。

7. 模块耦合。现象：`core/ontology.py` 单文件达 1380 行，是全仓最难维护的模块；同时 `core/` 与 `server/` 存在职责重叠——各自实现了 `disposal_board.py` 与 `graph_view.py`，存在逻辑双写（dual-write）风险。影响：本体编译器作为系统心脏，单体过大降低可测试性与可审查性；两端各写一份处置/图逻辑，则任一改动都可能因只改一处而产生行为不一致。建议：将 `core/ontology.py` 按编译子阶段（装载/物化/代理键/版本）拆分为子模块，保持对外接口不

变；对 `core/` 与 `server/` 的重叠实现做单一真相源收敛，把图与处置的"计算真相"统一收归 `core/`，`server/` 仅做展示适配。

8.6 演进路线

以下演进路线为建议性规划，非既定发布承诺，旨在为技术债（8.5）提供有序的消化路径。各阶段的目标与 8.5 的缺陷一一对应，优先级依据"先止血、再增强、后生态"的原则排布。

短期（1~2 个迭代内，建议）： 优先解决会直接误导维护者与削弱可信度的问题。其一，补齐 8.5 第 1 项的文档漂移，统一以 `GROUPS` 注册表为准并加 CI 校验；其二，针对第 4 项扩充规则覆盖，尤其是时间维度的独立规则与行为维度扩展，使五维基线均衡；其三，建立"policies 覆盖率检查"——确保每新增对象/链接声明即强制补 `policies.json` 策略，从机制上守住 fail-closed 边界，避免运行时被静默拒绝后才发现。

中期（建议）： 聚焦算法深度与跨案能力。图算法方向优先落地 Louvain 社区发现 + 最短路径（回应第 2、3 项），使团伙识别从"人工看两跳"升级为"算法提示 + 人工确认"；跨案方向依托 `core/pack.py` 的 `PackManager` 扩展跨包查询与 `cross_pack_audit`，支撑多案件联合研判；性能方向对语义层增量重建做优化，把 `DEFAULT_BATCH_THRESHOLD` 等常量改为可配置，并借 `benchmarks` 组（REQ-045）量化收益。

长期（建议）： 走向声明式生态。以本体（ontology）为内核资产，逐步建设本体市场（本体包可分发复用）、规则模板库（检测判据可组合沉淀）、以及多机构协作能力（在 fail-closed 与审计链约束下安全共享声明与结论）。长期目标不是把内核越做越大，而是让"查什么、怎么判定、谁能看"持续以声明形式被社区与跨机构共建，内核只负责确定性地编译与执行。

8.7 工程约定与贡献指引

本节将前述约定沉淀为可操作的检查清单，供代码审计者与贡献者对照执行。所有条目均来自本内核的真实约束，部分直接来自 `AGENTS.md` 已记载的"已知坑"，目的在于降低新维护者的踩坑成本。

修改核心代码前的检查清单： 第一，若改动涉及写路径，必须确认改动经由 `ActionExecutor` 单一入口，未在任何角落直写 DuckDB；第二，若改动涉及对象/链接声明，必须在 `policies.json` 同步补策略，否则运行时会被 fail-closed 拒绝；第三，若改动触及语义层编译，必须重跑 `validate_ontology.py --strict` 与 `build_ontology.py` 验证；第四，提交前本地执行 `run_tests.py`（非 `--fast`）确保全量测试组通过，且 `audit` 与 `guard` 两条静态红线组无违规。

0 已记载的已知坑（务必遵守）： 其一，`Store(db_path=":memory:")` 才是内存库，误用其他字符串会得到持久化库，造成测试污染；其二，建图必须"先节点后边"，节点必须全部导入，否则 COPY 边会报 "Unable to find primary key value"；其三，`lineage.prioritize_clues()` 必须接收其返回值，忽略返回值意味着优先级排序未被采用；其四，处置状态改完后必须重新生成 report，否则前端展示与持久化表会出现不一致。这些坑看似零散，但每一个都对应一次真实的故障。

新增声明的必填项： 新增对象需在 `objects.json` 声明 pk、kind、name_property 与 properties 值类型；新增链接需在 `links.json` 声明端点与边属性、在 `bindings.json` 提供 `build_sql`；新增规则必须在 `rules.json` 写 `rule_text`（自然语言判据）并绑定唯一 Function；任何新增能力若需新数据源，须走第 4 章的六步接入路径，且全程不修改检测器 Python 代码。

测试组注册规则： 新增功能必须绑定需求编号并在 `run_tests.py` 的 `GROUPS` 注册表中登记对应组，组名需与需求编号或能力名可对应；若该能力涉及降级路径，须归入 REQ-G 系列的"统一降级协议"分组，遵循"降级但不静默、留痕但不污

染"的断言约定：严禁在 AGENTS.md 等文档中硬编码测试组数量，所有组数引用须以 GROUPS 注册表为唯一事实来源。这一规则直接回应 8.5 第 1 项，是防止文档再次漂移的制度保障。
